Imam Mohammad Ibn Saud Islamic University

College of Computer and Information Sciences

Computer Science Department

Mirqab

A Smart Potholes Detection System For Drone Footage

By

Rayan Mazen Alharbi	444000557
Abdullah Rajeh Alshehri	444001478
Ziyad Abdulaziz Almoneef	444003721
Mazen Saleh AlAbdulatif	444001320

Supervised by:
Dr. Saad Alabbad

Abstract

Potholes and other forms of road damage impact transportation safety and lead to costly repairs, yet most road inspections are still performed manually, making them slow, labor intensive, and often unreliable. This project aims to develop an automated pothole detection system that combines drone imaging, computer vision, and GPS technology. High-resolution drone footage is processed using a deep learning model to identify potholes, while GPS data is used to record their exact geographic locations.

The system includes four main components: a drone-based data collection method, a detection model capable of analyzing images with high accuracy, a synchronized storage system linking detections with GPS coordinates, and an administrator dashboard for reviewing detections on an interactive map. Together, these components provide a faster and more efficient way to monitor road conditions.

The overall goal of the project is to reduce the need for manual inspections and support municipalities in maintaining safer road networks. By automating both detection and location reporting, the proposed system offers a practical and scalable solution for modern road maintenance challenges.

Table of Contents

1. **Chapter 1: Introduction**
   1.1 Problem Definition
   1.2 Aim of the Project
   1.3 Team Qualifications
   1.4 Objectives
   1.5 Conclusion
2. **Chapter 2: Literature Review**
   2.1 Introduction
   2.2 Background
   2.3 Related Works
   2.4 Conclusion
3. **Chapter 3: Requirements Analysis and Specification**
   3.1 User Requirements
   3.2 System Requirements
   3.3 System Static Architecture
   3.4 Use Case Diagram (Description)
   3.5 Project Management Plan
4. **Chapter 4: System Design**
   4.1 System Architecture
   4.2 Database Design
   4.3 Modular Decomposition
   4.4 System Organisation and Internal Behaviour
   4.5 Algorithms
   4.6 Alternative Designs and Justification
   4.7 Graphical User Interface Design
5. **Chapter 5: Implementation**
   5.1 Implementation Requirements
   5.2 Implementation Details
   5.3 Challenges Encountered and Solutions
6. **Chapter 6: Testing and Validation**
   6.1 Testing Strategy
   6.2 Test Cases
   6.3 Test Results and Discussion
7. **Chapter 7: Conclusion and Future Work**
   7.1 Summary
   7.2 Limitations
   7.3 Future Work
8. **Appendix**

CHAPTER 1

Introduction

Potholes and other street damage cause vehicle damage, cause accidents, and increase repair bills. Right now, most inspections are still done by hand, which takes a lot of time and isn't always reliable, especially when large areas need to be monitored. As cities grow and road networks expand, these traditional inspection methods become even less practical.

Recent studies have begun utilizing computer vision and deep learning to automate the detection of road damage. While these methods demonstrate significant potential, many existing systems lack the capability to accurately geolocate defects. Without precise GPS data, maintenance teams face challenges in responding rapidly and planning repairs effectively.

This project addresses this gap by integrating computer vision with geospatial data. The proposed system utilizes drone-captured footage to identify potholes and instantly correlates detections with precise GPS coordinates. This ensures that every identified defect is mapped, stored, and accessible via a centralized dashboard.

The goal is simple: enhance the efficiency and reliability of road monitoring. By automating both detection and location tracking, the system provides timely, actionable data, thereby improving road safety and reducing the reliance on manual inspections.

In this chapter, we introduce the background of the project, review existing research, and explain how the proposed system is designed to address the limitations found in current approaches.

1.1 Problem Definition

The primary challenge addressed by this project is the inefficiency of manual road surveillance. Automating this process requires overcoming specific technical constraints related to image analysis and geospatial tracking.

A core difficulty lies in designing a robust computer vision system capable of accurately locating potholes from a moving drone. The model must maintain high performance despite environmental variations, including uneven lighting, deep shadows, wet pavement, and changing camera viewpoints.

Furthermore, precise geolocation is critical. The system must implement methods to reliably synchronize visual detections with GPS data. This integration must account for latency between image processing and location logging, ensuring that the final reported coordinates are accurate enough for maintenance teams to locate and repair the damage.

1.2 Project Aim:

This project covers the design and implementation of an intelligent computer vision system to identify any street damage or potholes in particular and report their geographic locations. The deliverable will be a functional prototype to support municipalities and road maintenance authorities in monitoring street conditions more effectively, which means less time for manual inspection and lower maintenance costs, while improving road safety.

1.3 Objectives:

The primary aim of this project is to create an effective, automated system for road damage detection. The specific objectives are as follows:

- **To develop a deep learning model:** Design and train a computer vision model capable of distinguishing potholes from other road irregularities with high precision.
- **To integrate geospatial tracking:** Implement a synchronization mechanism that links visual detections with GPS coordinates, ensuring maintenance teams can accurately locate defects.
- **To establish a data acquisition pipeline:** Utilize drone-mounted cameras to capture consistent, high-resolution aerial footage of road surfaces for both model training and operational inference.
- **To design a visualization platform:** Build a centralized database and an interactive dashboard to store detection results and present them on a map, providing stakeholders with actionable insights.
- **To evaluate system performance:** Assess the system based on detection accuracy, operational efficiency, and feasibility under real-world conditions to validate its readiness for deployment.

1.3 Team Qualifications

Student Name	Technical Skills	Relevant Experience
Rayan Alharbi	Python, Java, PyTorch, TensorFlow, LangChain	AI, Machine Learning, Deep Learning, LLMs & Agents
Abdullah Alshehri	Java, React, Node.js, AWS, Git	Internship in full-stack development, Web app portfolio
Ziyad Almoneef	Python, ML, Deep learning, Tensorflow	Multiple Course projects in Deep Learning and Computer Vision
Mazen Alabdulatif	Python, JavaScript, Docker, SQL, Cloud	Software engineering, web app development, CI/CD, Drones

1.5 Conclusion

In this chapter, the motivation for the project was set in terms of the huge negative impact that street damage and holes in particular have on both road safety and maintenance costs. Current approaches to inspection are manual, slow, and inefficient and hence not appropriate for large-scale monitoring.

The proposed solution addresses these limitations by designing an automated system that integrates computer vision with GPS technology to detect damages and record their precise locations. Key challenges identified include ensuring detection reliability across varying environmental conditions and achieving precise synchronization between visual defects and geospatial coordinates.

To overcome these challenges, the project outlines a methodology that includes a high-accuracy detection model (e.g., YOLO), drone-based data acquisition, and a centralized dashboard for visualization. The system will be evaluated based on its accuracy, operational efficiency, and feasibility for real-world deployment.

By fulfilling these objectives, the project aims to provide a practical, scalable solution that supports more efficient road maintenance practices and enhances overall transportation safety.

CHAPTER 2

Literature Review

2.1 Introduction

The previous chapter outlined the problem of road damage and motivated the need for an automated pothole detection system. It also highlighted the importance of early detection in enhancing road safety, reducing vehicle maintenance costs, and minimizing the expense of large scale repairs.

This chapter examines the existing research, methods, and technologies related to pothole detection. It begins by providing essential background information to clarify the significance of the problem and to introduce the technological foundations of modern detection systems. It then reviews a range of studies that have approached the problem using different techniques, comparing their methodologies, strengths, and limitations.

The objectives of this chapter are to:

- Explain the relevance of pothole detection within the broader field of smart transportation.
- Describe the primary technologies and approaches commonly used in pothole detection systems.
- Review previous research efforts, analyze their methods, and highlight both their contributions and remaining gaps.

2.2 Background

2.2.1 Importance of the Problem

Good road conditions are crucial for safe and efficient transportation. Of the various types of road damage, potholes are probably the most common and frustrating. They develop whenever, over time, surface materials weaken due to environmental factors like changes in temperature, water absorption, and repeated heavy traffic loads.

Undetected and unrepaired potholes may result in traffic accidents, damage to vehicles, and congestion. The classical inspection methods involve on-site road surveying, which is clearly very labor-intensive and expensive. These are often associated with the risks of having workers in unsafe environments. In contrast, the use of drones has provided a faster, safer, and more productive way of monitoring the roads. Drones can cover large areas quickly and capture high-resolution imagery of road surfaces; hence, they can easily detect potholes with high accuracy and speed. This would, of course, enable the building up of smart transportation systems.

2.2.2 Evolution of Pothole Detection Research

Pothole detection has undergone considerable change in research due to the growth of technological capabilities. Indeed, previous work relied on either manual inspection or vehicle-mounted systems where road conditions were recorded, either using onboard cameras or simple sensors. While these methods were very informative, they were again slow, restricted to a very small area coverage, and labor-intensive.

Methods have evolved with the development of computer vision, where several edge-detection, thresholding, and contour extraction image-processing techniques have been applied on road images to identify potholes.

Even though these methods outperformed the traditional approaches presented above, their performance was very sensitive towards changing light and shadow conditions, road texture, and possible debris; hence, they are not very reliable under practical scenarios.

With the increase in computational power, machine learning techniques like Support Vector Machines and Random Forest classifiers became conventional. These models were dependent on hand-engineered features such as color, texture, and shape descriptors that typically enhanced the detection accuracy. The success of these models was still very dependent upon the quality of the hand-engineered features.

Recently, this methodology has been revolutionized by deep learning. It enables features to be learned in a self-sufficient way directly from raw images with either CNNs or even more advanced object-detection models, hence allowing for more robust and more accurate detection results.

Finally, deep learning combined with drone-based imaging enables large-scale, high-resolution monitoring of road surfaces from an aerial perspective. Drone systems capture long road segments in a fast and efficient way, while offering a larger field of view compared to traditional cameras located on the ground or mounted on vehicles.

This progression from manual surveys to AI-driven aerial detection describes the development of increasingly faster, more scalable, and more reliable solutions for pothole monitoring.

2.2.3 Key Concepts and Technologies

This section describes some of the key concepts and technologies that make up the basic framework of drone-based pothole detection for efficient and precise analysis of road surfaces. These drones, in other words Unmanned Aerial Vehicles, would be a suitable platform for data acquisition because they are capable of capturing detailed images of the road surface from various heights and angles using high-resolution cameras, while recording the precise location of such images with the addition of GPS modules.

Computer vision is central to such interpretation. As a subfield of AI explicitly concerned with the handling of visual data, computer vision provides methods for identifying imperfections or damage on road surfaces. It is in this regard that deep learning, neural network models enable the automation of such detection through learning patterns directly from large datasets of annotated images.

Additionally, geolocation further refines detection by linking each identified pothole with its exact geographic coordinates. Spatial information is thus available for effective mapping, reporting, and follow-up maintenance activities. Captured images may be processed onboard via edge-computing hardware within the drone or sent to remote servers for cloud-based analysis according to system requirements and the computational resources at hand.

Coupled together, these technologies are the key enablers of a state-of-the-art drone-based road inspection system combining scalability with intelligence in the assessment of road conditions.

2.2.4 Current Challenges

Although drone-based pothole detection has made significant progress, several challenges still affect the accuracy and practicality of real-world deployments.

One major issue is the variability in image quality. Drone footage can suffer from motion blur, shadows, glare, or low lighting depending on weather, time of day, and flight altitude. These factors make it difficult for computer vision models to consistently identify potholes, especially when road surfaces have similar textures or when damage appears subtle.

Models perform best when trained on large and diverse datasets, but many available pothole datasets are limited or captured under very specific conditions. As a result, models may not generalize well to new environments, different road materials, or unfamiliar lighting situations.

Achieving precise GPS adds a further layer of complexity. Even when a pothole is accurately detected in an image, matching it with the correct GPS coordinate is not always straightforward. GPS inaccuracy can lead to location mismatches that reduce the usefulness of detection results for maintenance teams.

There are also operational constraints, drones have limited battery life.High-resolution images and videos, while necessary for detection accuracy, demand substantial storage capacity and processing power. Managing this data efficiently both onboard the drone and during later analysis is crucial for smooth system performance.

In summary, while drone-based pothole detection shows strong potential, real-world conditions introduce challenges related to image quality, model generalization, geolocation precision, and operational limitations. Addressing these issues is essential for building a reliable, scalable, and practical system.

2.3 Related Works

[1] HemaMalini B.H., Akshay Padesur, V. Manoj, *"Detection of Potholes on Roads using a Drone,"* 2018.

[2] Young-Mok Kim, Young-Gil Kim, Seung-Yong Son, *"Review of Recent Automated Pothole-Detection Methods,"* 2022.

[3] Arpit Kumar Bhatt, Hasumati Raj, Vinamra Sharma, *"Advancements in Pothole Detection Techniques: A Comprehensive Review and Comparative Analysis,"* 2025.

[4] Yashar Safyari, Masoud Mahdianpari, Hodjat Shiri, *"A Review of Vision-Based Pothole Detection Methods Using Computer Vision and Machine Learning,"* 2023.

[5] Muhammad Haroon Asad, Saran Khaliq, Muhammed Harron Yousaf, Muhammad Obaid Ullah, Afaq Ahmad, *"Pothole Detection Using Deep Learning: A Real-Time and AI-on-the-Edge Perspective,"* 2023.

Table 1 Overview of Related Studies

Ref. No.	Approach / Method	Data Source	Main Technique	Key Findings	Strengths	Limitations
[1]	Drone-based image processing	Aerial road images (UAV)	YOLOv3 algorithm and GPS	Detected potholes effectively when filled with water	Low cost, simple setup	Slow and inefficient, need for accurate differentiation
[2]	Review of automated detection methods	Multiple published studies	Vision & Vibration based methods	High accuracy rates in vision based	Comparison of multiple systems	Limited pothole data
[3]	Comparative review of detection techniques	Multiple prior works	Vision and Sensor based	Deep learning enhances overall accuracy	Covers many approaches	Sensors affected by other factors
[4]	Vision-based ML review	Public & local datasets	2d & 3d, and deep learning	Hybrid approach is robust	Combination of approaches	Increased complexity
[5]	Deep learning & edge AI	Online sources	Real time detection	Achieved real-time and accurate detection	Fast, edge deployable	Must equip on vehicles

In summary, the reviewed studies present the history of how pothole detection evolved from basic image processing to advanced models of deep learning.

The early approaches, such as [1], focused on simple edge detection and shape analysis, which indeed worked in controlled conditions but failed under poor lighting.

Later studies such as [4] and [5] used deep learning models such as YOLO, respectively, and reached real-time, accurate detection.

Recent reviews [2, 3] point out that deep learning yields excellent results, although faced high computational demands. A drone-based approach, such as in [1], is very promising because flexible, large area inspections become possible, but its accuracy depends on the visual conditions and model quality.

The identified research gap entails a drone-based pothole detection system that integrates deep learning into lightweight with real-time performance that could fit deployment in the real world.

2.4 Conclusion

The chapter reviewed the scientific basis of pothole detection and major recent research advances. It highlighted early detection, the need for automation, and computer vision and machine learning as some of the key technologies involved.

A review of related works showed that, especially with drone-based imaging, deep learning models can provide the most accurate and efficient results in detecting potholes. On the other hand, there are challenges that still need to be overcome by existing systems, including sensitivity to environmental conditions, limited datasets, and hardware constraints.

These limitations point to a clear research gap, which the current project is set to fill, hence the basis for developing a drone-based pothole detection system using deep learning for reliable, accurate, and scalable road monitoring.

CHAPTER 3

Requirements Analysis and Specification

The objective of this chapter is to define and document all requirements of the proposed drone-based pothole detection system. Requirements analysis ensures that the system meets the needs of its intended users namely data collectors (field operators) and administrators (maintenance supervisors).

This chapter describes the functional and non-functional requirements, the system architecture, the system-level specifications derived from user needs, and the project management plan. Proper specification is essential before moving to design and implementation, as it ensures clarity, consistency, and project feasibility.

3.1 User Requirements

To identify the system requirements, several activities were conducted:

- **Field Observation:** Reviewing how municipalities currently conduct road inspections.
- **Expert Consultation:** Discussions with an experienced drone engineer who provided practical insights into drone operations.
- **Document analysis:** Examination of prior research and existing pothole detection systems to understand typical limitations and user expectations.

These activities helped identify what the end users (data collectors and administrators) need from the system.

3.1.1 User Functional Requirements:

The functional requirements are defined to ensure the system effectively supports the workflow of both Data Collectors and Administrators:

- **FR-01 (Data Acquisition):** The Data Collector must be able to capture high-resolution aerial footage of road surfaces using the drone platform.
- **FR-02 (Automated Detection):** The system must automatically process the captured footage to identify potholes using computer vision algorithms.
- **FR-03 (Geospatial Logging):** The system must synchronize valid detections with their precise GPS coordinates and store this metadata in a centralized database.
- **FR-04 (Interactive Visualization):** The Administrator must be able to view detected defects plotted as markers on an interactive map interface.
- **FR-05 (Quality Control):** The system must provide a mechanism for the Administrator to flag or validate detections (False/True Positives) to support continuous model improvement.
- **FR-06 (Reporting):** The system must allow the exportation of detailed detection reports to assist in maintenance planning.

3.1.2 User Non-Functional Requirements

To ensure the system is practical, safe, and efficient, the following non-functional constraints are defined:

- **NFR-01:** The administrative dashboard must be intuitive and user friendly, allowing personnel with minimal technical expertise to navigate and utilize the system effectively.
- **NFR-02:** The system must process uploaded footage and populate the dashboard with detection data with minimal latency to ensure timely reporting.
- **NFR-03:** The drone operations must not disrupt active traffic flow or pose a safety risk to vehicles and pedestrians.
- **NFR-04:** The hardware components must be easily transportable to facilitate inspection of diverse geographic locations.
- **NFR-05:** The web-based dashboard must be responsive and accessible via standard web browsers across both desktop and mobile devices.

3.2 System Requirements

System requirements provide a deeper and more detailed specification derived from the user requirements.

3.2.1 System Functional Requirements

1. **Drone Data Capture Module**
    - **SFR-01:** The system shall capture high-resolution footage during drone flights.
    - **SFR-02:** The system shall associate captured imagery with the closest available GPS data.
2. **Pothole Detection Module**
    - **SFR-03:** The system shall run the object detection model on collected footage.
    - **SFR-04:** The system shall detect potholes and produce their bounding boxes.
3. **Data Storage & Synchronization Module**
    - **SFR-05:** The system shall synchronize timestamps between images and GPS logs.
    - **SFR-06:** The system shall store detections, and GPS coordinates.
4. **Administrator Dashboard Module**
    - **SFR-07:** The system shall display detections on an interactive map.
    - **SFR-08:** The system shall allow administrators to flag incorrect detections.
    - **SFR-09:** The system shall generate exportable reports.

3.2.2 System Non-Functional Requirements

The following quality attributes define the operational boundaries and performance targets of the system:

Performance Requirements

- **SNFR-01:** The system shall perform offline inference on the full batch of collected footage and produce detection results within 24 hours of data upload.
- **SNFR-02:** The system shall load the dashboard map within 3 **seconds** under normal network conditions.

Accuracy Requirements

- **SNFR-03:** The deep learning model shall achieve a mean Average Precision (mAP) of at least **85%** under optimal daylight conditions.
- **SNFR-05:** The geospatial logging shall maintain a location accuracy within ±3 – 7 meters.

Security Requirements

- **SNFR-05:** Access to the administrative dashboard shall be restricted to authenticated users via a secure login mechanism.
- **SNFR-06:** All data transmission between the client and the server must be encrypted using the HTTPS (TLS/SSL) protocol.

Scalability Requirements

- **SNFR-07:** The database must support storing **at least 10,000 detections**.
- **SNFR-08:** The system backend shall be designed to support data ingestion from multiple drone sources simultaneously for future expansion.

Reliability Requirements

- **SNFR-09:** The system shall implement local data buffering to ensure no detection data is lost if internet connectivity is interrupted during upload.
- **SNFR-10:** The web dashboard shall maintain an availability uptime of at least **95%** during operational hours

3.3 System Static Architecture

The system is divided into four primary subsystems. Each subsystem handles a specific stage of the pothole detection pipeline, from data capture to visualization and management.

1. **Drone Subsystem**
    - Camera
    - GPS module
    - Onboard storage or transmission unit
2. **Image Processing and Detection Subsystem**
    - Preprocessing unit
    - YOLO-based detector
    - Timestamp synchronizer
3. **Backend & Storage Subsystem**
    - Detection database
    - Metadata storage
    - API endpoints

4. **Administrator Interface Subsystem**
    - Interactive map
    - Detection viewer
    - Flagging and reporting tools

These subsystems interact to complete the detection cycle from data collection to final review.

Figure 1 System architecture

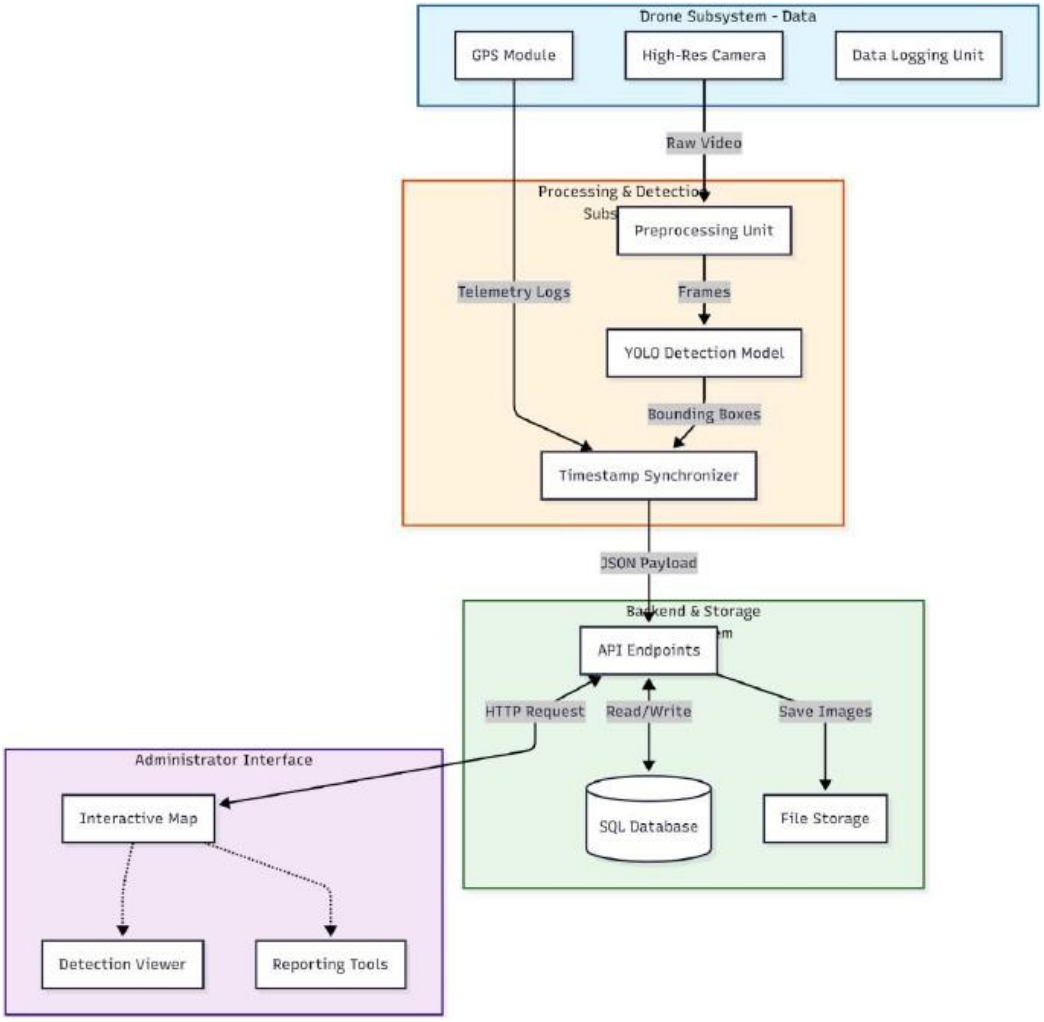
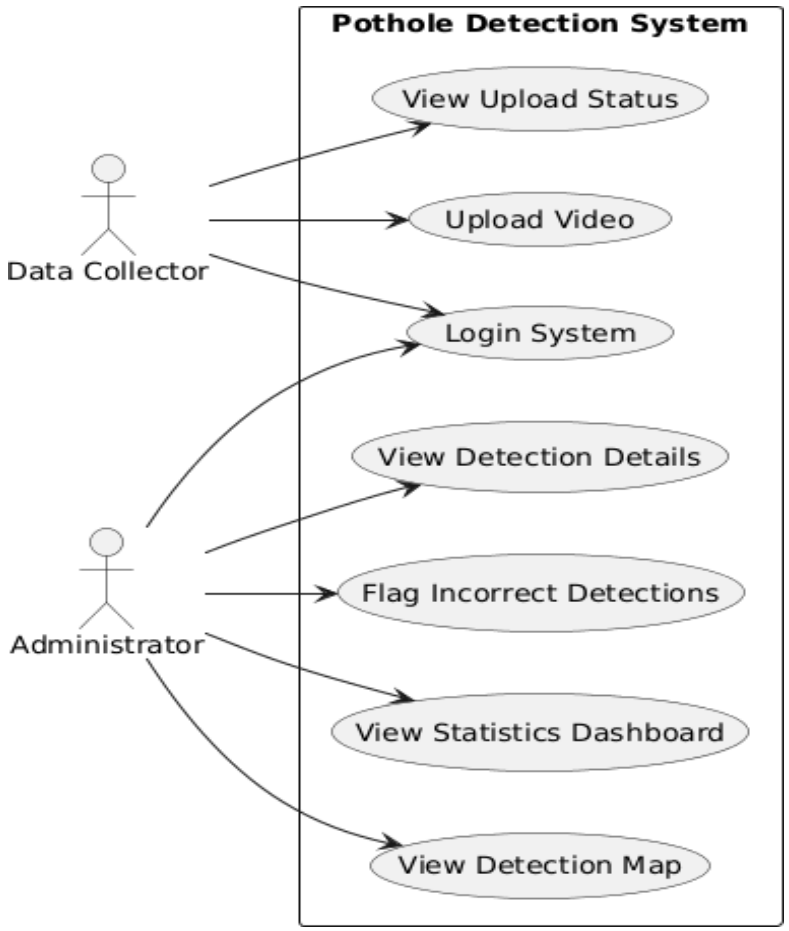


Figure 2 use case diagram



3.4 Use Case Diagram Description

Use Case 1 — Upload Video

Use Case Attribute	Description
Use Case ID	UC01
Use Case Name	Upload Video
Primary Actor	Data Collector
Importance Level	High
Type	Essential
Description	The Data Collector uploads drone-captured footage for pothole detection.
Trigger	Data Collector finishes drone flight and wants to submit the footage.

Type	Condition
Preconditions	The Data Collector is logged in.
	A valid video file is available.
	Internet connection is active.
Postconditions	Video is uploaded and stored.
	Metadata is recorded.
	File is queued for AI processing.

Normal Flow

Step	Action
1	User opens the Upload Page.
2	User selects video file.

3	System validates file format and size.
4	System uploads the video.
5	System confirms successful upload.

Alternative / Exception Flow

Case	Description
a	Invalid file format → Error displayed.
b	Network failure → Upload paused or user prompted to retry.
c	Storage failure → System logs error and notifies user.

Use Case 2 — View Upload Status

Use Case Attribute	Description
Use Case ID	UC02
Use Case Name	View Upload Status
Primary Actor	Data Collector
Importance Level	Medium
Type	Essential
Description	The Data Collector checks the status of uploaded videos
Trigger	User wants to track video processing progress.

Type	Condition / Outcome
Preconditions	User must be logged in.
Preconditions	At least one upload must exist.
Postconditions	User sees the current processing status.

Normal Flow

Step	Action
1	User opens the Upload Status page.
2	System retrieves status information.
3	System displays progress, timestamps, or errors.

Alternative / Exception Flow

Case	Description
a	No uploads found → System displays appropriate message.

Use Case 3 — Login System

Use Case Attribute	Description
Use Case ID	UC03
Use Case Name	Login System
Primary Actor	Data Collector / Administrator
Importance Level	High
Type	Essential
Description	Allows authenticated access to system based on user roles.
Trigger	User enters login credentials.

Type	Condition / Outcome
Preconditions	User has a valid registered account.
Postconditions	Successful authentication and dashboard access.

Normal Flow

Step	Action

1	User inputs username and password.
2	System verifies credentials.
3	System redirects user to respective dashboard.

Alternative / Exception Flow

Case	Description
a	Invalid credentials → System displays “Incorrect username or password”.
b	Account disabled → System notifies the user.

Use Case 4 — View Detection Details

Use Case Attribute	Description
Use Case ID	UC04
Use Case Name	View Detection Details
Primary Actor	Administrator
Importance Level	High
Type	Essential
Description	Admin reviews detection details such as images, GPS.
Trigger	Administrator opens detection records.

Type	Condition / Outcome
Preconditions	Administrator is logged in.
Preconditions	Detection records exist.
Postconditions	Administrator views detection details.

Normal Flow

Step	Action
1	Administrator opens Detection Details page.
2	System retrieves detection data.
3	System displays images, GPS data, and confidence scores.

Alternative / Exception Flow

Case	Description
a	No detection data → System displays message.

Use Case 5 — Flag Incorrect Detections

Use Case Attribute	Description
Use Case ID	UC05
Use Case Name	Flag Incorrect Detections
Primary Actor	Administrator
Importance Level	High
Type	Essential
Description	Administrator marks false detections to improve system accuracy.
Trigger	Admin identifies a wrong detection.

Type	Condition / Outcome
Preconditions	Admin logged in.
Preconditions	Detection record exists.
Postconditions	Detection marked as incorrect for future model refinement.

Normal Flow

Step	Action
1	Admin selects detection record.
2	Admin clicks “Flag Incorrect”.
3	System updates detection status.

Alternative / Exception Flow

Case	Description
a	Already-flagged record → System displays warning.

Use Case 6 — View Statistics Dashboard

Use Case Attribute	Description
Use Case ID	UC06
Use Case Name	View Statistics Dashboard
Primary Actor	Administrator
Importance Level	Medium
Type	Essential
Description	Shows statistics such as detection counts, severity, dates, and trends.
Trigger	Administrator opens dashboard.

Type	Condition / Outcome
Preconditions	Administrator is logged in.
Postconditions	Dashboard displayed with summarized analytics.

Use Case 7 — View Detection Map

Use Case Attribute	Description
Use Case ID	UC07
Use Case Name	View Detection Map
Primary Actor	Administrator
Importance Level	High
Type	Essential
Description	Displays pothole locations on an interactive GPS map.
Trigger	Administrator wants to view pothole distribution.

Type	Condition
Preconditions	Detection data must include GPS coordinates.
	Admin must be authenticated.
Postconditions	Map loaded with plotted potholes.

Normal Flow

Step	Action
1	Admin opens Detection Map page.
2	System loads map and coordinates.
3	Pothole markers appear on the map.

Alternative / Exception Flow

Case	Description
a	Missing GPS → System labels detection as “Unknown location”.

Figure 3

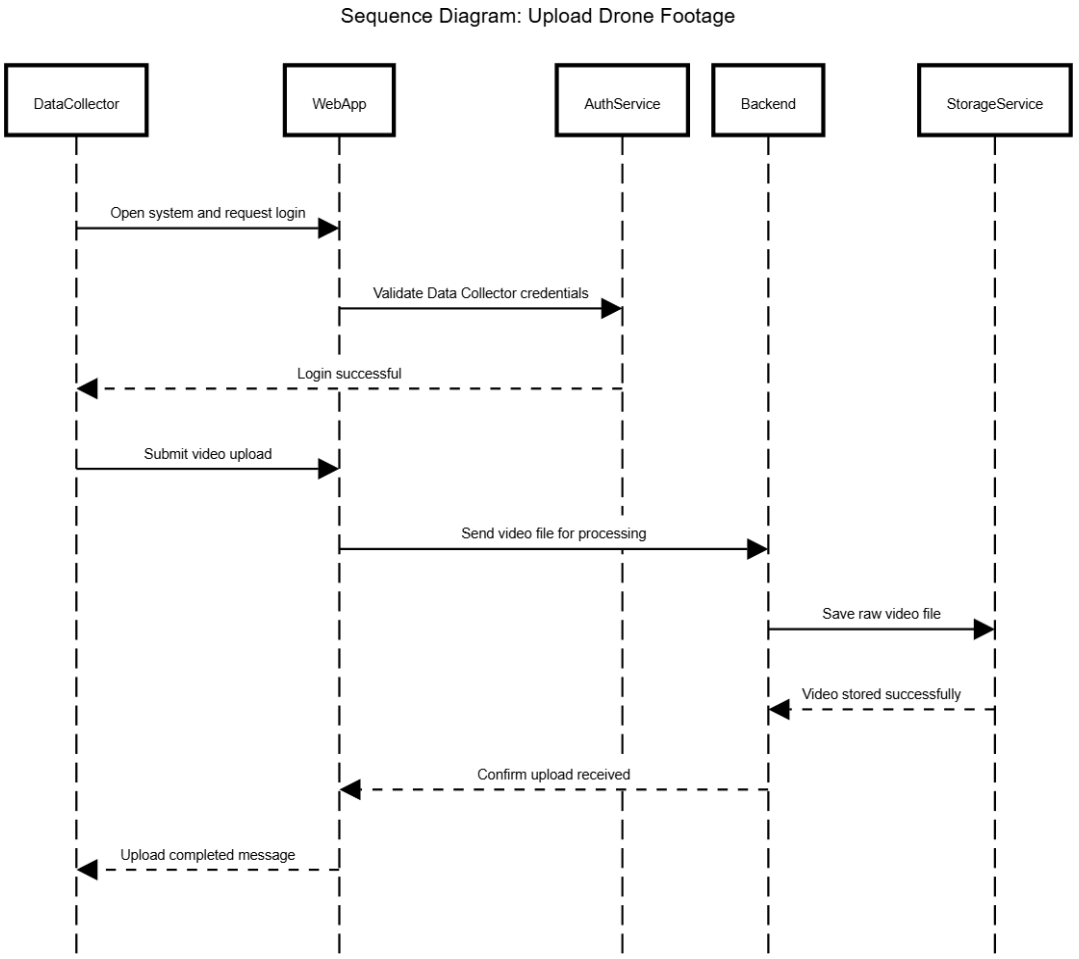

Sequence Diagram: Upload Drone Footage
DataCollector
WebApp
AuthService
Backend
StorageService
Open system and request login
Validate Data Collector credentials
Login successful
Submit video upload
Send video file for processing
Save raw video file
Video stored successfully
Confirm upload received
Upload completed message

Figure 4

Sequence Diagram: YOLO Detection Processing

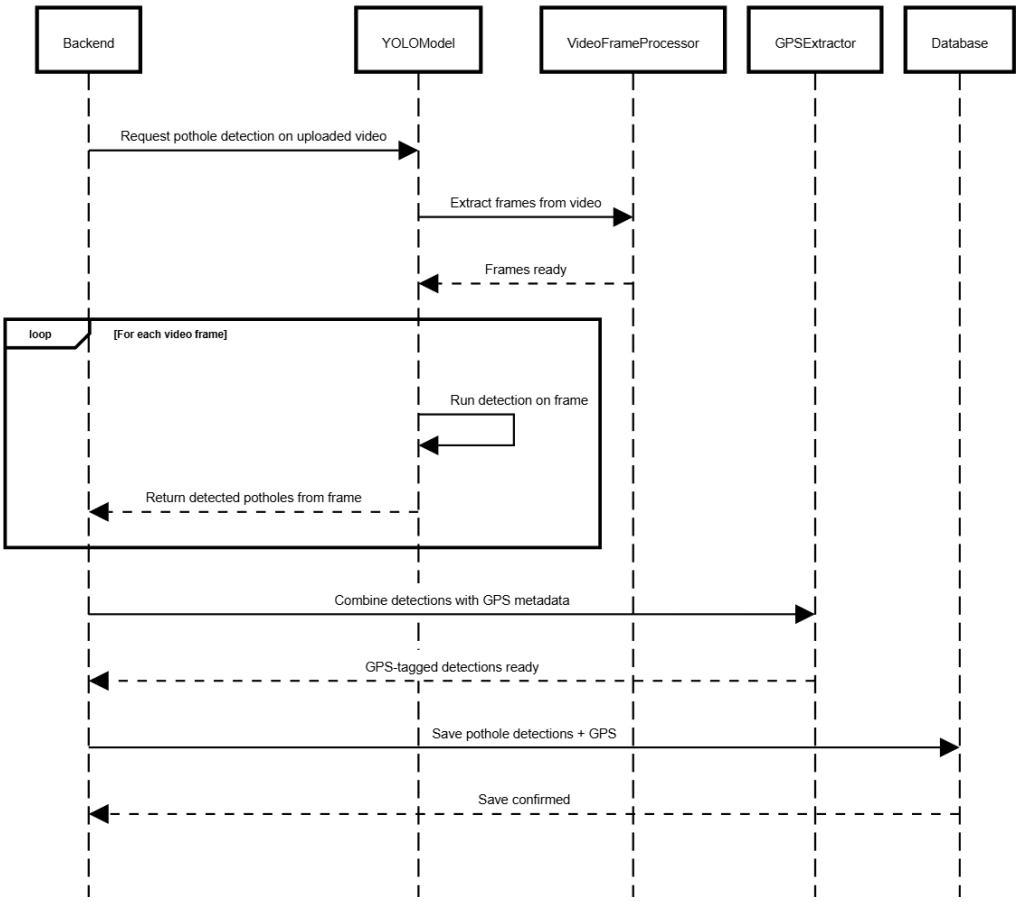
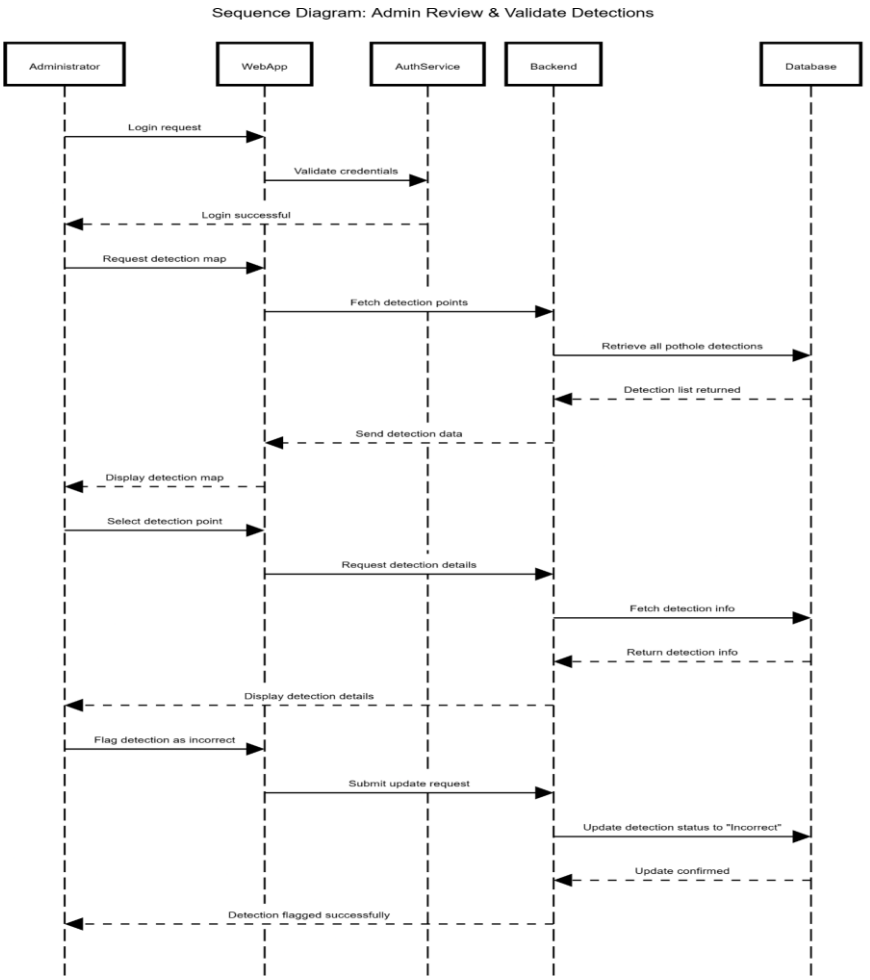
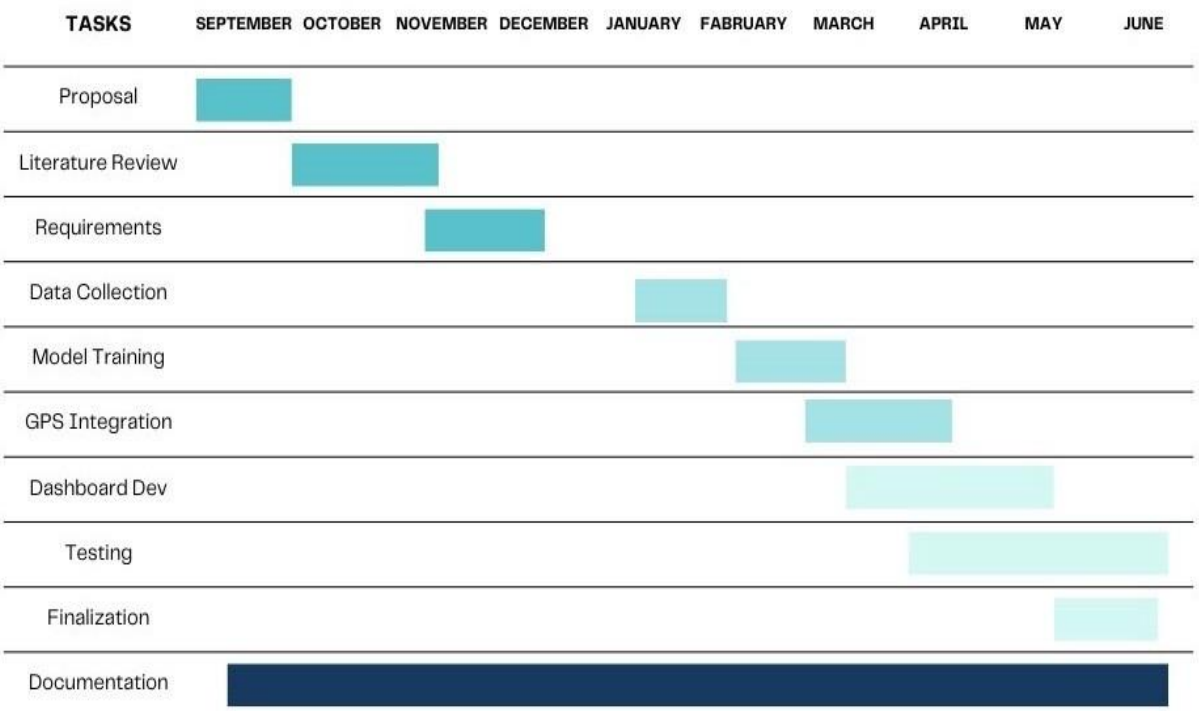


Figure 5


Sequence Diagram: Admin Review & Validate Detections
Administrator
WebApp
AuthService
Backend
Database
Login request
Validate credentials
Login successful
Request detection map
Fetch detection points
Retrieve all pothole detections
Detection list returned
Send detection data
Display detection map
Select detection point
Request detection details
Fetch detection info
Return detection info
Display detection details
Flag detection as incorrect
Submit update request
Update detection status to "Incorrect"
Update confirmed
Detection flagged successfully

3.5 Project Plan



CHAPTER 4

System Design

This chapter presents the architectural design of the Mirqab system, converting the requirements defined in Chapter 3 into a technical blueprint, the design decisions documented here are justified against the functional and non-functional requirements and evaluated relative to feasible alternatives, we will cover the overall system architecture, database schema, modular decomposition, internal behavioural models, algorithms, alternative design, and GUI.

4.1 System Architecture

The system follows architectures comprising a data acquisition tier, a processing tier, and a presentation tier and each tier maps directly to a principal subsystem, and the tiers communicate through defined interfaces.

4.1.1 Subsystem Overview

The system is organised into three principal subsystems, each responsible for a distinct stage of the detection pipeline:

- Inference Pipeline: This subsystem contains the computer vision core and it accepts drone captured video and applies a trained object detection model to identify potholes and produces annotated image frames together with a detection record file and the second stage has optical character recognition to extract GPS coordinates from the drone display overlay and merges them into the detection record.
- Web Application: This subsystem serves as both the RESTful API server and the static-file host for the frontend and it is structured according to a layered architectural pattern and each layer depending only on the layer beneath it via a TypeScript interface.
- Frontend Dashboard: The presentation layer provides an interactive web based dashboard comprising a login screen, a map view built on the Leaflet library, a sortable and filterable detections table, a video upload interface with progress feedback, and an administrative review screen for validating or rejecting individual detections.

4.1.2 Communication Patterns

The subsystems interact through the following architectural patterns:

- Client–Server: The browser communicates with the Express API exclusively through JSON RESTful endpoints.
- Layered Architecture: Within the web application layers are separated by TypeScript interfaces, ensuring that each layer depends only on its neighbour.
- Pipe and Filter: Raw video frames pass through the detection model then through the OCR based GPS extractor.
- Repository: Data access is abstracted behind a contract interface with interchangeable implementations, a PostgreSQL/PostGIS-backed repository for production and a file-backed JSON repository for development.

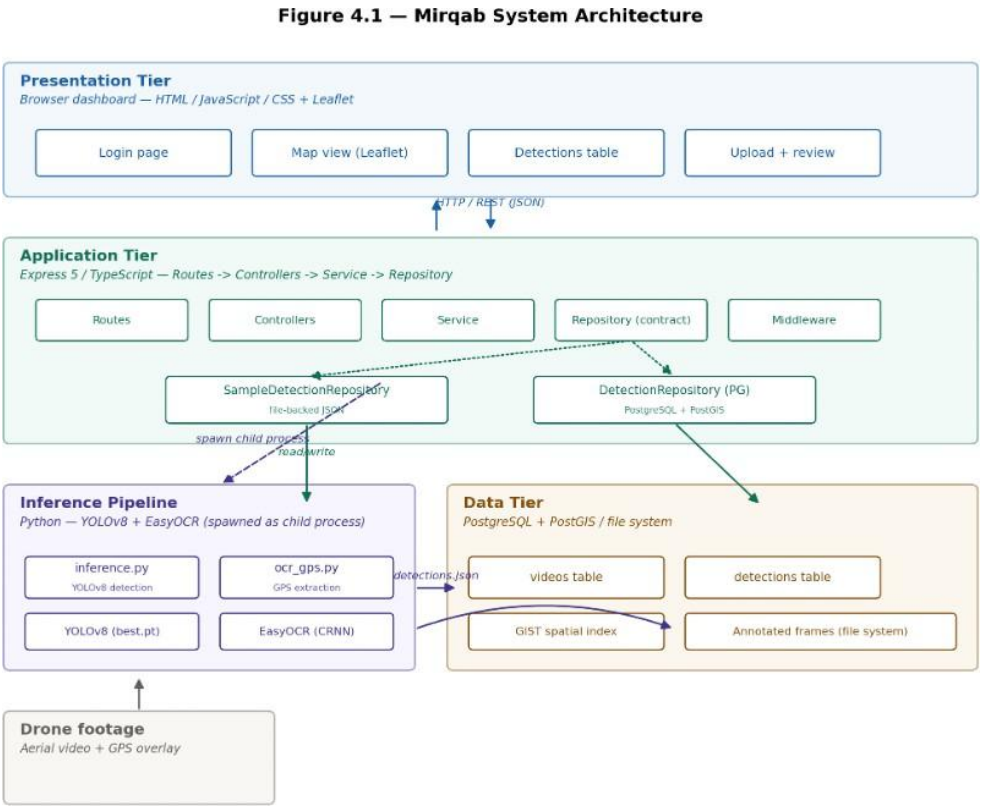


Figure 6: Mirqab architecture, showing the four tiers and their communication patterns.

4.2 Database Design

The persistent data layer uses PostgreSQL with the PostGIS extension to support geographic queries and the relational schema consists of two principal entities connected by a one-to-many relationship.

4.2.1 Entity Descriptions

Videos: Each record represents a single drone flight video submitted for analysis and attributes include a surrogate primary key, a unique business identifier, the original source file path, the capture start timestamp, the video duration, and an automatic creation timestamp.

Detections: Each record represents an individual pothole identified in a specific video frame and attributes include a surrogate primary key, a unique detection identifier, a foreign key referencing the parent video, the frame index, the video timestamp in seconds, a confidence score constrained between zero and one, geographic coordinates stored both as separate numeric columns and as a PostGIS geographic point for spatial indexing, the file path to the annotated frame image, an optional review status, and a review timestamp.

4.2.2 Relationships and Constraints

The schema enforces referential integrity through a foreign key from detections to videos with cascading deletion, the check constraints validate that confidence values fall within the unit interval, frame indices are non-negative, and geographic coordinates are between valid ranges.

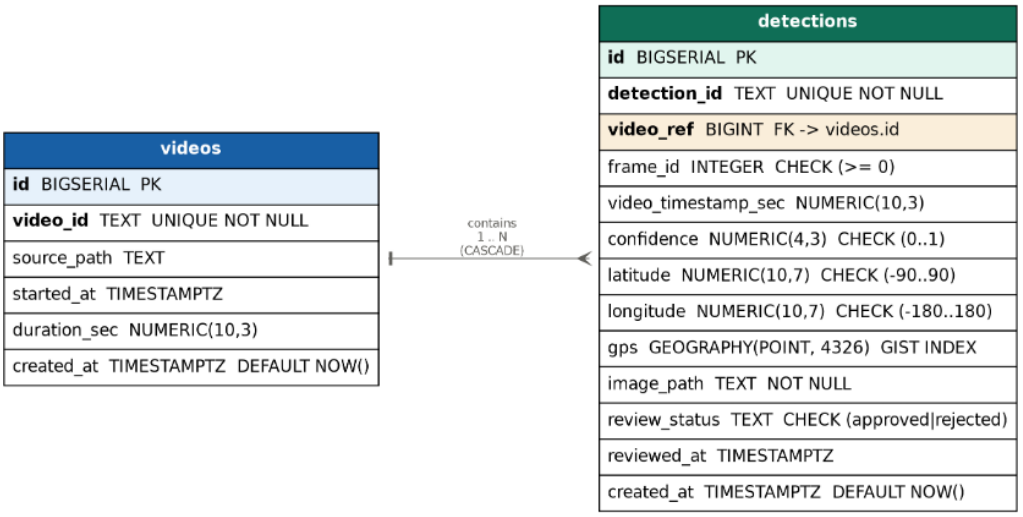


Figure 7: Entity–Relationship diagram for Mirqab database schema.

4.3 Modular Decomposition

The system employs a hybrid decomposition strategy, combining object oriented decomposition for the web application with data-flow decomposition for the computer vision pipeline.

4.3.1 Object-Oriented

The backend is breakdown into the following principal classes and interfaces:

- DetectionService:

  The service layer encapsulates all business logic, including listing videos, querying detections with filters, computing aggregate statistics, retrieving map data, and managing administrative reviews.

- DetectionRepositoryContract:

  An interface implemented by three concrete repositories:

  DetectionRepository (PostgreSQL/PostGIS).

  SampleDetectionRepository (file-backed JSON with in-memory review state).

  DynamicDetectionRepository (substitution of the active repository)

- Controllers: Factory functions that receive a service instance and return Express request handlers, covering detections, authentication, health checks, and the pipeline upload workflow.
- Middleware. Authentication guards (session based), input validation (Zod schemas), and a centralised error handler mapping application level exceptions to standardised HTTP error responses.

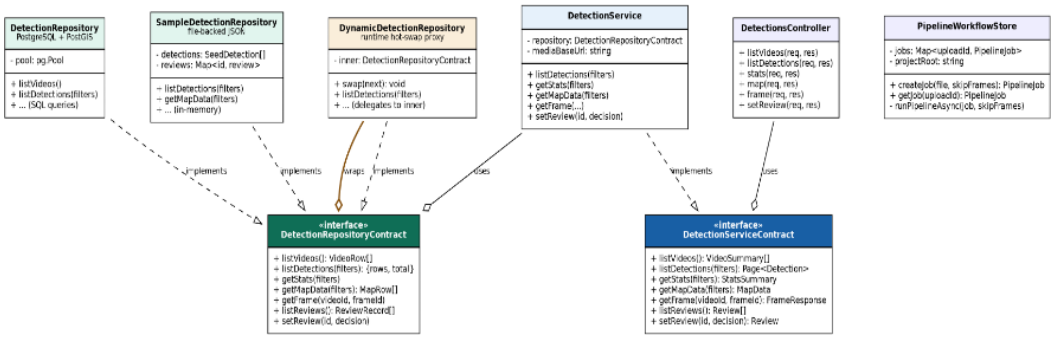


Figure 8: UML class diagram showing the service, repository, and controller relationships.

4.3.2 Data Flow (Inference Pipeline)

The Python pipeline follows a sequential data flow model with six stages:

1. Frame iteration with configurable skip frame sampling.
2. YOLOv8 object detection on each sampled frame.
3. Annotated frame rendering and output video generation.
4. Detection record construction into a structured JSON file.
5. GPS extraction from raw frames using EasyOCR on the drone overlay, validated against a trusted anchor track built from cluster-OCR.
6. Coordinate merging into the detection records.

Each stage operates as a pure transformation and reading its input from the file system and writes its output to a designated directory. This design ensures that partial results are preserved if a stage fails and that individual stages can be re-run independently.

4.4 System Organisation and Internal Behaviour

This section describes the dynamic behaviour of the system through sequence and state models.

4.4.1 Video Upload and Detection Sequence

The upload workflow proceeds as follows:

User submits a drone video through the dashboard upload form, the Express server receives the file via a multipart upload handler, persists it to the upload directory, and creates a pipeline job entry, the server immediately returns a job identifier to the client, which begins polling for progress updates. The pipeline workflow store spawns the Python inference script as a child process, as the script processes frames, it emits progress indicators to standard output, which the store parses and exposes through the polling endpoint.

Upon successful completion of inference, the store automatically spawns the GPS extraction script, once both stages finish the store marks the job as complete, constructs a new file-backed repository from the output, and hot-swaps it into the dynamic repository so that the dashboard reflects the new results without a server restart.

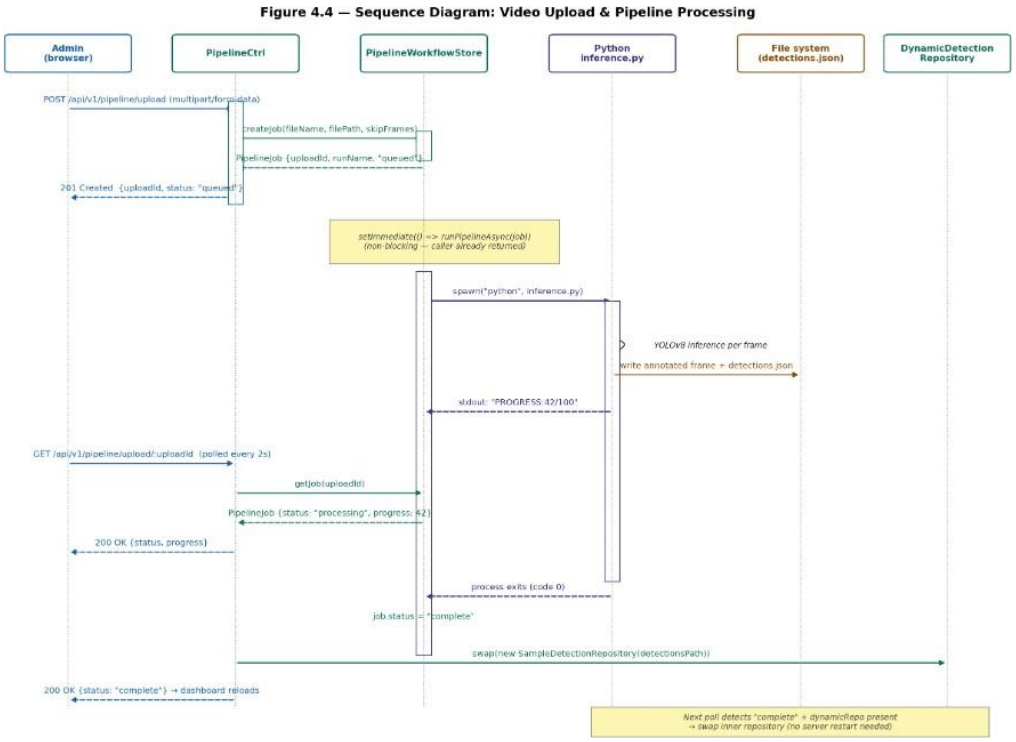


Figure 9: Sequence diagram for the video upload and pipeline processing workflow.

4.4.2 Pipeline Job State Model

Each pipeline job transitions through a defined set of states: Queued (initial), Inference (processing frames), GPS (OCR extraction stage is running), Complete (both stages succeeded), and Failed (an error occurred during either stage). Transitions from Inference and GPS to Failed are triggered by a non-zero exit code from the respective Python process.

4.4.3 Dataflow Diagram

At the highest level of abstraction, the dataflow diagram traces a drone video file through inference (producing annotated frames and an initial detection file), then through GPS extraction (enriching detections with coordinates), into the repository and service layers, and finally to the dashboard for visualisation.

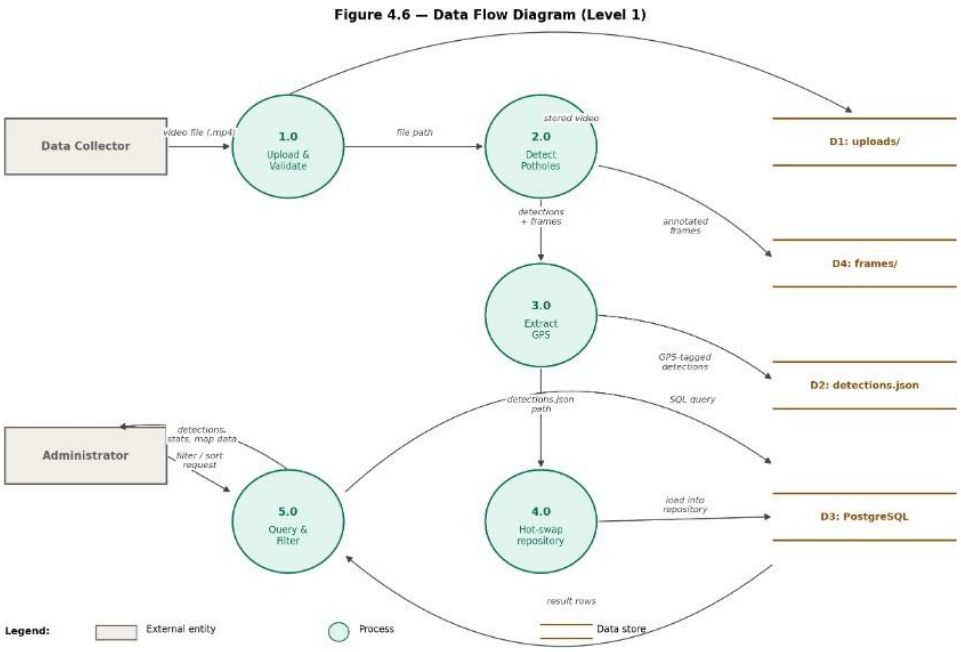


Figure 11: Level-1 dataflow diagram for the Mirqab pipeline.

4.5 Algorithms

The following sections provide the pseudocode of the algorithms that form the basis of the system's detection and data handling logic.

4.5.1 Algorithm 1: Video Inference with Frame Skipping

This algorithm iterates through a video and applies the YOLOv8 model to sampled frames and saves annotated outputs, and accumulates detection records. Frame skipping is employed to reduce computational cost while maintaining adequate temporal coverage.

Input: model M, video V, skip interval S, confidence C *Output:* detection records D, video, frames

1. Open video stream from V
2. Compute total frames T and frames to process $P = \lceil T / S \rceil$
3. Initialise empty list D
4. For each frame F at index i in the video stream:
    5. If $S > 1$ and $(i \bmod S) \neq 1$, write F to output video and skip
    6. Run M.predict(F, confidence = C) to obtain result R
    7. Draw bounding boxes on F using R; write annotated frame to output video
    8. If R contains detections, save annotated frame as image and append each detection (with frame index, timestamp, confidence, null GPS) to D
    9. Emit progress indicator
10. Flush D to output file and return

4.5.2 Algorithm 2: GPS Extraction from Drone On-Screen Display

This algorithm extracts geographic coordinates from the drone's overlay text using optical character recognition and it processes two cropped regions of each frame (top-left for longitude, top-right for latitude) and applies a robust parsing function that tolerates OCR noise.

Input: frame image F, *Output:* (latitude, longitude) or null if unreadable

1. Crop the top-left band of F as region R_lon
2. Crop the top-right band of F as region R_lat
3. Apply OCR to R_lon and R_lat to obtain text segments
4. For each set of segments, concatenate all digit characters, discarding non-numeric noise
5. If the resulting digit string has fewer than six characters, return null
6. Return the parsed (latitude, longitude) pair

4.5.3 Algorithm 3: Repository Hot-Swap After Upload Completion

This algorithm ensures that the dashboard reflects newly processed data without requiring a server restart. When the polling endpoint detects that a pipeline job has completed, it constructs a new repository instance from the output file and atomically replaces the active repository reference.

Trigger: client polls GET /api/v1/pipeline/upload/{id}

1. Retrieve job J by upload identifier
2. If J.status = "complete" and J has not been swapped:
    3. Construct a new file-backed repository R from J.detectionsPath
    4. Call dynamicRepository.swap(R) to atomically replace the active data source
    5. Mark J as swapped
6. Return current job status and progress to client

4.6 Alternative Designs and Justification

Several design alternatives were evaluated during the architectural planning phase. This section documents the key decisions and the rationale for each choice.

4.6.1 Detection Model Selection

Three candidate architectures were considered for the pothole detection model. Faster R-CNN offers higher accuracy on small objects but operates slower than real time single shot detectors, rendering it impractical for processing drone video at scale. YOLOv5 represented a viable baseline, however YOLOv8 improves upon it with an anchor-free detection head, enhanced default data augmentation, and a more streamlined Python API through the Ultralytics package.

More recent variants such as YOLOv9 and YOLOv10 were also considered; however, their adoption would require additional integration effort and deviation from the stable Ultralytics

pipeline used in this project, given that YOLOv8 already satisfied the performance and accuracy requirements for drone based pothole detection, the marginal benefits of newer architectures didn't justify the increased implementation complexity.

Therefore, YOLOv8 was selected as the optimal trade-off between inference speed and detection accuracy for single-GPU deployment.

4.6.3 Monolithic vs. Microservice Architecture

A microservice architecture separating the Node.js API and the Python inference pipeline into independently deployed services communicating over gRPC was considered. The monolithic approach in which the API spawns the Python pipeline as a child process, was selected because it eliminates inter service network complexity, simplifies deployment to a single application, and requires only a single repository and environment configuration. The interface between the two components constitutes a well defined seam that could be replaced with a network boundary in a future iteration if horizontal scaling becomes necessary.

4.6.4 Data Layer Strategy

The repository pattern supports two interchangeable implementations. PostgreSQL with PostGIS provides production grade spatial indexing, concurrent write support, and efficient geographic queries. The file backed JSON repository enables local development and demonstration without requiring database installation. The DetectionRepositoryContract interface ensures that all application logic remains agnostic to the underlying storage mechanism.

4.6.5 Frontend Framework

Singlepage application frameworks such as React and Vue were evaluated for the dashboard. Given that the interface comprises a small number of screens with moderate interactivity, vanilla JavaScript served over static HTML was selected. This choice eliminates the need for a build toolchain, reduces deployment complexity, and avoids introducing framework specific dependencies for a component that does not warrant them.

4.7 Graphical User Interface Design

The dashboard interface is designed as a right-to-left Arabic-language web application and comprises the following principal screens:

- Login Page: A minimal authentication form accepting username and password credentials, with inline error feedback for invalid attempts.
- Upload: The first operational screen containing an upload card (file selector with a skip-frames parameter and a live progress bar)
- Dashboard: An interactive Leaflet map displaying colour-coded detection markers by confidence level, and a sortable, filterable, paginated detections table.
- Administrative Review: Controls allowing the administrator to approve or reject individual detections, supporting iterative quality assurance and future model refinement.

CHAPTER 5

Implementation

This chapter documents the technical realisation of the design specified in Chapter 4. It covers the hardware and software prerequisite, the development tools and their justification, key implementation details including data structures and selected procedures, the graphical user interface, and the significant technical challenges that faced us during development.

5.1 Implementation Requirements

5.1.1 Hardware

The system was developed and tested under two hardware configurations. The minimum configuration suitable for demonstration and development on CPU-only machines, requires an x86-64 processor with at least four cores, eight gigabytes of RAM, and five gigabytes of free disk space for model weights, annotated frames, and uploaded videos. The recommended configuration for training and fast inference adds a GPU with at least six gigabytes of VRAM, sixteen gigabytes of system RAM, and twenty gigabytes of solid-state storage.

The hardware choices are driven by the computational demands of the detection model: YOLOv8 inference on a five-minute 1080p video with a skip interval of thirty frames requires approximately two minutes on CPU but completes in seconds on a modern GPU. The EasyOCR model similarly benefits from GPU acceleration.

A drone with an on screen GPS display is required for data acquisition; the system was tested with DJI footage.

5.1.2 Software

The software environment comprises Node.js 20 or later, Python 3.10 or later (for the YOLOv8 and EasyOCR inference pipeline), and optionally PostgreSQL 14 or later with PostGIS 3 or later for the production data store. CUDA 11.8 with cuDNN is required only for GPU-accelerated inference. Git LFS is used to manage the versioned model weights file.

5.1.3 Programming Languages

Three languages are used, each selected for specific domain:

- TypeScript (backend):

  Static typing provides compile-time verification of the contract interfaces between the service, repository, and controller layers.

- Python (inference pipeline):

  Python is the most used language for machine learning, and the only language with first class support for the Ultralytics, OpenCV, EasyOCR, and PyTorch libraries required by the pipeline.

- HTML, JavaScript, and CSS (frontend):

  The dashboard's limited scope does not justify introducing a framework build toolchain; vanilla JavaScript served as static files minimises deployment friction.

- SQL (PostgreSQL with PostGIS). Used for schema definition, migration scripts, and spatial queries on the detection data.

5.1.4 Tools and Technologies

Domain	Tool Selected	Justification
Object Detection	Ultralytics YOLOv8	Superior speed–accuracy trade-off compared to Faster R-CNN, anchor-free head improves small-object performance.
OCR	EasyOCR	Deeplearning CRNN architecture outperforms Tesseract.
Web Framework	Express 5	Mature ecosystem with extensive middleware support.
Database	PostgreSQL + PostGIS	Best-in-class open-source spatial database, GIST indexing enables efficient bounding-box and radius queries in logarithmic time.
Mapping	Leaflet	Lightweight open-source web mapping library with a rich plugin ecosystem.
Validation	Zod	Schema-first validation with TypeScript type inference, eliminates runtime type mismatches at the API boundary.
Testing	Node test runner + Supertest	Zero additional dependencies; ships with Node.js 20; Supertest provides HTTP-level assertions.

Table 2 Principal tools and technologies with justification.

5.2 Implementation Details

5.2.1 Deployment and Installation

The installation procedure consist of six steps: installing Node.js 20 and Python 3.10, cloning the repository with Git to fetch the model weights, installing Node.js dependencies, creating a Python virtual environment and installing the pipeline requirements, copying the environment configuration template, and optionally configuring PostgreSQL with PostGIS and running the database migration scripts. The application starts with single command (npm start) and is accessible at http://localhost:3000.

5.2.2 Data Structures

The primary persistent data structure are the two relational tables, videos and detections, described in Section 4.2. GPS coordinates are stored as separate numeric latitude and longitude columns for simple filtering and response serialization, and the database schema also includes a PostGIS geography point column with a GiST index for spatial queries.

In sample data mode, the SampleDetectionRepository hold the parsed detection array in memory and maintains a separate Map structure keyed by detection identifier for administrative review state. The DynamicDetectionRepository wraps a swappable inner repository reference, enabling the server to replace its active data source at runtime after a new upload is processed.

The PipelineWorkflowStore maintains a Map of upload identifiers to job records, each tracking the job status (queued, inference, gps, complete, or failed), current progress, any error or warning messages, and the path to the output detections file.

Each detection record in the JSON output also includes gps_source string field alongside the gps dictionary, It takes the value “ocr” when the coordinate was read directly from the frame and validated, “interpolated” when recovered from the anchor track,

5.2.3 Key Procedures

This section provides a brief description of the key procedure implemented in the system. Table 2 summarises each procedure, its source file, and its role. Two short code excerpts are included below the table to illustrate the most architecturally distinctive logic. Complete source code is provided in the Appendix.

Procedure	Location	Purpose
run_video_inference	pipeline/scripts /inference.py	Reads video frames with skip-N sampling, runs YOLOv8 detection, writes annotated output and a detections JSON file incrementally.
_parse_segments	pipeline/scripts /ocr_gps.py	Parses multi-segment EasyOCR output into a valid geographic coordinate by stripping noise and testing 2- and 3-digit integer parts.
_collect_anchors	pipeline/scripts/ocr_gps.py	Builds a GPS anchor track from cluster-OCR and samples every 30s.
DynamicDetectionRepository.swap	src/repositories /dynamic-detection-repository.ts	Atomically replaces the inner data repository so the dashboard reflects new detection results without a server restart.
runPipelineAsync	src/services /pipeline-workflow-store.ts	Spawns the Python inference script as a child process, parses real-time progress from stdout, runs OCR-based GPS extraction, and transitions the job through processing to complete.
getUploadStatus	src/controllers /pipeline-workflow-controller.ts	Polls job status; on completion, constructs a new file-backed repository and hot-swaps it into the dashboard.

Table 3 Summary of key procedures.

Inference Pipeline (run_video_inference)

This procedure opens the input video and iterate every frame. It also supports skip frames for faster inference. For each processed frame, detected bounding boxes are recorded with a unique ID, timestamp, and confidence score. The detection JSON file is rewritten after every frame so that partial results survive if the process is interrupted. Progress lines are printed to standard output in a format that the Node.js parent process parses to update the job progress percentage in real time.

GPS Coordinate Parsing (_parse_segments)

EasyOCR often returns the on-screen GPS overlay as multiple fragmented text blocks rather than a single string. The function strips all non-digit characters, concatenates the remaining digits then splits them into two parts, The code is shown below:

```
def _parse_segments(segments):
   digit_segments = []
          for seg in segments:
      cleaned = re.sub(r"[^0-9. ]", "", seg)
          if cleaned.strip():
         digit_segments.append(cleaned)
          if not digit_segments:
      return None
          digits = "".join(digit_segments).replace(" ", "").replace(".", "")
          if len(digits) < 6:
      return None
          for int_len in [2, 3]:
          if len(digits) > int_len:
         try:
           val = float(f"{digits[:int_len]}.{digits[int_len:]}")
         except ValueError:
           continue
        if -180 <= val <= 180:
          return val
          return None
```

Before processing detection frames, run_ocr build an anchor track by calling _collect_anchors, which cluster-OCRs and samples frames every thirty seconds. Each detection's OCR result is then checked against the nearest anchor's velocity cone and failed or rejected readings are recovered via interpolation from the anchor track.

Hot-Swap Repository (DynamicDetectionRepository)

This class implements Algorithm 3. It acts as a thin proxy that forward every data-access method to an inner repository instance. The single mutating method, swap(), atomically replaces that inner instance with a new one. When the inference pipeline completes and produces a new detections file, the controller constructs a fresh file-backed repository and calls swap(). From that moment on, every dashboard query reads from the latest run's data without requiring a server restart.

```
export class DynamicDetectionRepository
 implements DetectionRepositoryContract {

 constructor(private inner: DetectionRepositoryContract) {}

 swap(next: DetectionRepositoryContract): void {
  this.inner = next; } // All other methods delegate to this.inner }
```

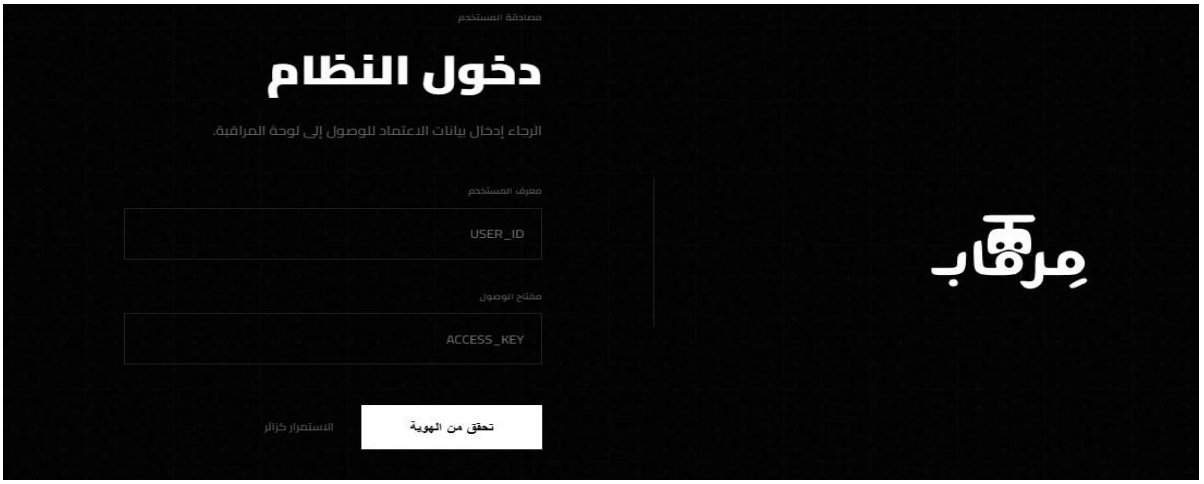
Pipeline Orchestration (runPipelineAsync & getUploadStatus)

These two procedure work together to manage the pipeline lifecycle. runPipelineAsync spawns the Python inference script as a child process and listens on its standard output for progress updates. The call is dispatched via setImmediate so that the originating HTTP

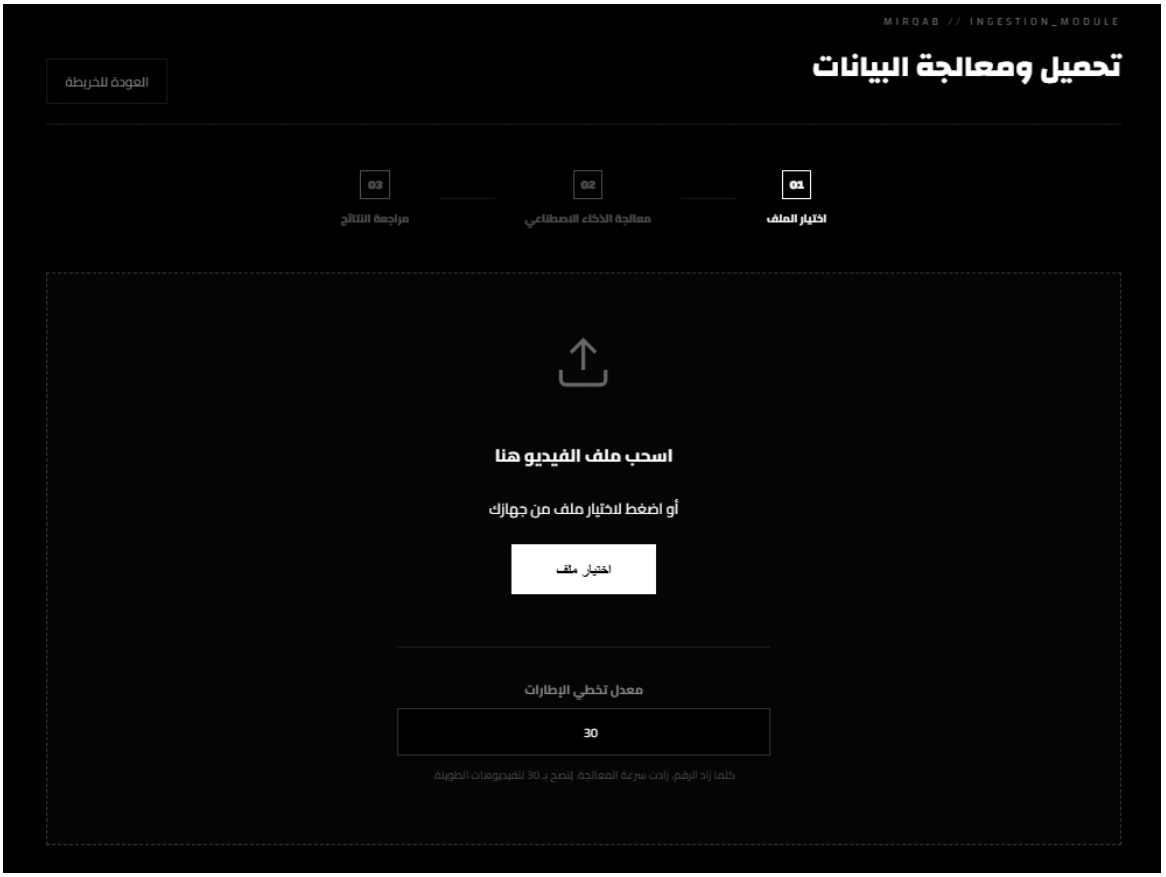
request returns immediately rather than blocking until inference finishes. getUploadStatus is the polling endpoint: each time the front end checks the job's status, the controller inspects whether the pipeline has completed. If it has, and the result has not yet been loaded, it constructs a new repository from the output JSON and calls the hot-swap mechanism described above. This design keeps the upload flow non-blocking and ensures the dashboard updates automatically.

5.2.4 Graphical User Interface

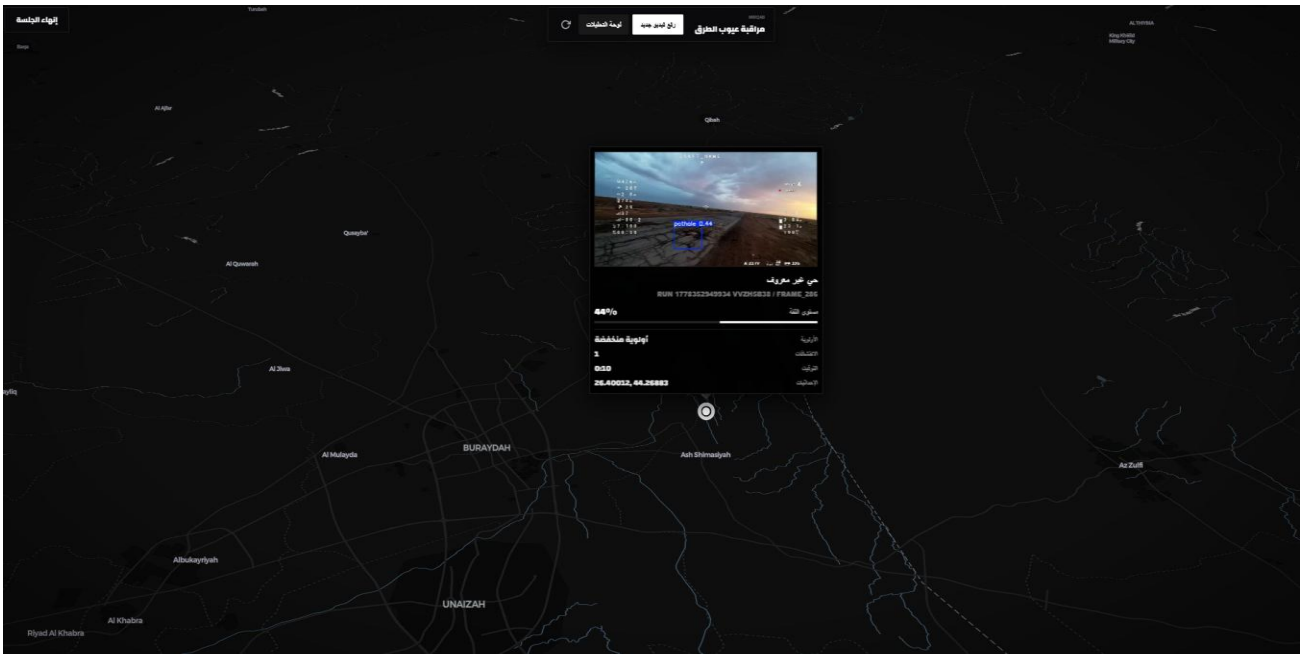
The login screen presents a form with username and password fields. Credentials are submitted as JSON to the authentication endpoint. On success, the server establishes a session cookie and grants access to the dashboard and upload workflow.



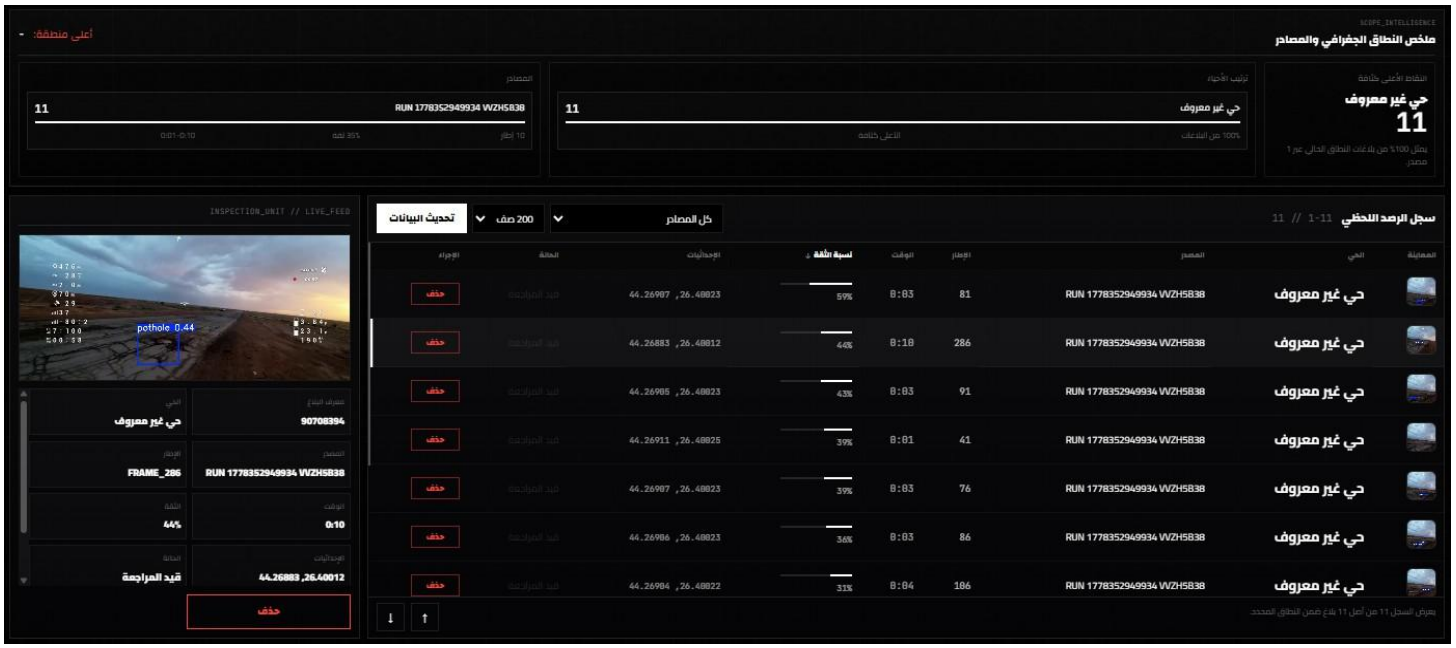
The upload screen is a separate protected workflow. It provides a video file picker/drop zone, a numeric input for the skip-frames parameter, and a progress interface. During processing, the frontend polls the pipeline status endpoint and updates the progress bar according to the job state.



The map screen uses MapLibre GL to display detection points returned by the map API. Markers are rendered from grouped GPS detection data, and selecting a marker opens a popup containing the annotated frame thumbnail, confidence score, frame metadata, timestamp, and coordinate.



The dashboard screen is the primary analytical and review interface. It displays summary telemetry, charts, filtering and pagination controls, a detections table, and an inspection panel with the annotated frame image. Administrative review controls allow authenticated users to mark detections as approved/restored or rejected/deleted as false positives.



5.3 Challenges Encountered and Solutions

Several significant technical challenges arose during implementation:

1- Extracting MetaData from drone

Our initial approach aimed to pull telemetry data directly from the drone. However, this has been proven to be technically infeasible. As an alternative, we implemented an OCR pipeline to extract the required metadata from the video feed, which displays flight parameters overlaid on screen by the Flight Controller.

2- OCR accuracy on drone overlay text

Past attempts used Tesseract OCR, which is optimised for printed text on clean backgrounds, Tesseract produced systematically incorrect digit readings. Replacing Tesseract with EasyOCR, which employs a deep learning CRNN resolved the issue.

3- Partial GPS coverage

A small number of frames had unreadable GPS overlays due to motion blur or glare, and some produced incorrect coordinates when EasyOCR misread a digit. This was resolved by introducing an anchor-track validation which is trusted sequence of GPS anchors built independently from original video, and each detection frame's OCR result is validated against the nearest anchor and frames that fail OCR are assigned coordinates via interpolation between bracketing anchors.

4- Long running inference blocking the HTTP response

The initial implementation waited for the entire pipeline to complete before returning a response to the upload request, causing the dashboard to appear frozen, the solution was to refactor the upload endpoint to return immediately with job identifier, offload the pipeline to a child process, and provide a polling endpoint that streams real time progress updates to the frontend.

CHAPTER 6

Testing and Validation

This chapter describes the testing strategy employed to verify that the implemented system satisfies the requirements defined in Chapter 3, the approach combines automated unit and integration testing of the web application with manual end-to-end testing of the full pipeline, from video upload through detection to dashboard display.

6.1 Testing Strategy

First at the unit level, the web application's service and controller layers are tested in isolation using a fake service implementation injected through the composition root's override mechanism and at the integration level, HTTP level tests exercise the full middleware stack (authentication, validation, error handling) against the fake service, and at the system level, manual tests verify the end-to-end pipeline including Python inference, GPS extraction, and dashboard visualisation.

The automated test suite uses Node.js's built-in test runner with the Supertest library for HTTP assertions and the createApp factory accepts a FakeService object, enabling the HTTP layer to be exercised without requiring a live database or the Python pipeline.

6.2 Test Cases

The following table summarises the test cases, each mapped to the requirement it validates. Detailed test implementations are provided in the Appendix.

ID	Description	Expected Result	Result
TC-01	Health endpoint	Returns 200 with status "ok"	Pass
TC-02	Unauthenticated access	Returns 401 JSON error	Pass
TC-03	Login and session round-trip	Session cookie set; authenticated endpoints return 200	Pass
TC-04	Login with invalid credentials	Returns 401 with standard error contract	Pass
TC-05	Filter detections by minimum confidence	All returned detections have confidence ≥ threshold	Pass
TC-06	Invalid query parameter	Returns 400 with validation details from Zod	Pass
TC-07	Statistics endpoint	Returns total detections, unique videos, and per-video aggregates	Pass

TC-08	Map data endpoint	Returns one entry per (video, frame) with aggregated confidence	Pass
TC-09	Frame endpoint (known frame)	Returns 200 with detections array	Pass
TC-10	Frame endpoint (unknown)	Returns 404 with “not_found” code	Pass
TC-11	Review: approve detection	Returns 200; subsequent query reflects approval	Pass
TC-12	Review: invalid decision value	Returns 400 with validation error	Pass
TC-13	Mock upload lifecycle	Progress increases monotonically; final status is “complete”	Pass
TC-14	Pipeline: reject non-video	Returns 400 for unsupported file type	Pass
TC-15	Pipeline: end-to-end upload	Dashboard displays new detections without server restart	Pass
TC-16	Inference on test video	detections.json contains at least one record; annotated frames exist	Pass
TC-17	OCR coordinate parsing	parse_segments returns correct decimal coordinate	Pass
TC-18	Repository hot-swap	New run’s detections appear	Pass

Table 4: Summary of test cases with requirement traceability and results.

6.3 Test Results and Discussion

All eighteen test cases passed in the final testing cycle and during earlier iterations, several failures were identified and corrected:

- TC-03 initially failed because the session middleware was not correctly configured for login requests; adding the appropriate content-type handling resolved the issue.
- TC-15 revealed the stale-data problem described in Section 5.3, which led to the development of the DynamicDetectionRepository hot-swap mechanism.
- TC-17 exposed edge cases in the OCR parser where EasyOCR returned fewer than six digits for heavily blurred frames; the algorithm was updated to create an anchor track and validate OCR result.

The test suite provides confidence that the core functional requirements operate correctly under both normal and error conditions. Manual testing of TC-15, TC-16, and TC-18 confirmed that the full pipeline, from raw drone video to dashboard visualisation, functions as specified.

CHAPTER 7

Conclusion and Future Work

7.1 Summary

This project aimed to design and implement an automated system for detecting potholes in aerial drone video and reporting their geographic locations, reducing reliance on manual inspections and supporting safer road maintenance.The system achieved the objectives defined in Chapter 1:

- **Deep learning model.** A model was trained and integrated into a video-processing pipeline capable of frame by frame pothole detection with configurable sampling.
- **Geospatial tracking.** A GPS extraction stage uses EasyOCR-based OCR to recover coordinates from the drone's on-screen display, validated against a trusted anchor track to reject misreads and recover coordinates for unreadable frames via interpolation.
- **Data acquisition pipeline.** The system accepts drone video uploads through a web interface and processes them through an automated inference and GPS extraction pipeline with real-time progress updates.
- **Visualisation platform.** A Node.js/TypeScript dashboard combines a REST API, MapLibre GL, filterable detections table, and administrative review workflow within a browser authenticated interface.
- **Performance evaluation.** The system was evaluated using eighteen unit, integration, and end-to-end test cases. All tests passed in the final cycle, confirming that functional and non-functional requirements were met under the tested conditions.

7.2 Limitations

- GPS coverage depends on the readability of the drone's on-screen display. Anchor track interpolation recovers coordinates for most frames where direct OCR fails, while the anchor track improves the GPS extraction it also depends on the OCR.
- The Python pipeline runs as a single child process, meaning that only one video can be processed at a time; concurrent uploads queue implicitly but are not load-balanced across multiple workers.
- The session store is held in memory; a server restart logs out all users, and the system cannot be horizontally scaled in its current form.
- There is no user management interface; administrative credentials are configured through environment variables.
- The OCR parser is calibrated for a specific drone on-screen display layout (longitude in the top-left corner, latitude in the top-right); other drone manufacturers or firmware versions would require additional calibration.
- The detection model classifies all damage as a single "pothole" class and does not distinguish severity levels.

7.3 Future Work

The following enhancements are proposed for future iterations of the system:

- Multi user authentication and role based access control. Implementing a persistent users table with hashed passwords and distinct roles (administrator, reviewer, read-only) to support multi operator deployment.
- Scalable job processing. Introducing a persistent session store (e.g., Redis) and a message-queue-based job dispatcher to enable horizontal scaling of the web server and parallel processing of multiple video uploads on GPU equipped hosts.
- Native drone telemetry integration. Parsing the drone's native telemetry sources when available, retaining OCR as a fallback for re-encoded footage lacking sidecar files.
- Audit trail and export capabilities. Recording the full review history for each detection (reviewer identity, timestamp, comments) and supporting export of approved detections as GeoJSON or Shapefile for integration with municipal asset-management systems.
- Severity classification. Extending the detection model from a binary pothole/non-pothole classifier to a multi-class model distinguishing severity tiers (minor, moderate, severe), enabling maintenance teams to prioritise repairs.
- Edge inference. Deploying a quantised version of the detection model on a drone companion computer (e.g., NVIDIA Jetson Nano) to perform on-device inference and transmit only detection records to the server, reducing bandwidth requirements.

References

[1] B. H. HemaMalini, A. Padesur, and V. Manoj, "Detection of Potholes on Roads using a Drone," in Proceedings of the International Conference on Recent Trends in Engineering and Technology, 2018.

[2] Y.-M. Kim, Y.-G. Kim, and S.-Y. Son, "Review of Recent Automated Pothole-Detection Methods," Applied Sciences, vol. 12, no. 11, 5320, 2022.

[3] A. K. Bhatt, H. Raj, and V. Sharma, "Advancements in Pothole Detection Techniques: A Comprehensive Review and Comparative Analysis," International Journal of Transportation Science and Technology, 2025.

[4] Y. Safyari, M. Mahdianpari, and H. Shiri, "A Review of Vision-Based Pothole Detection Methods Using Computer Vision and Machine Learning," Sensors, vol. 23, no. 12, 5575, 2023.

[5] M. H. Asad, S. Khaliq, M. H. Yousaf, M. O. Ullah, and A. Ahmad, "Pothole Detection Using Deep Learning: A Real-Time and AI-on-the-Edge Perspective," Advances in Civil Engineering, 2023.

[6] G. Jocher et al., "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[7] JaidedAI, "EasyOCR, Ready-to-use OCR with 80+ supported languages," GitHub repository, 2024. [Online]. Available: https://github.com/JaidedAI/EasyOCR

[8] PostGIS Development Group, "PostGIS, Spatial and Geographic Objects for PostgreSQL," 2024. [Online]. Available: https://postgis.net

[9] V. Agafonkin, "Leaflet, an open-source JavaScript library for mobile-friendly interactive maps," 2024. [Online]. Available: https://leafletjs.com

[10] OpenJS Foundation, "Express, Fast, unopinionated, minimalist web framework for Node.js," 2024. [Online]. Available: https://expressjs.com

[11] C. McMahon, "Zod, TypeScript-first schema validation with static type inference," 2024. [Online]. Available: https://zod.dev

Appendix

This appendix contains the complete source code for the most architecturally significant files of the Mirqab project. The full project repository is available at: https://github.com/AbdullahRajeh/Mirqab

Appendix A: Database Schema

Source: sql/migrations/001_init.sql

```sql
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE IF NOT EXISTS videos (
  id BIGSERIAL PRIMARY KEY,
  video_id TEXT NOT NULL UNIQUE,
  source_path TEXT,
  started_at TIMESTAMPTZ,
  duration_sec NUMERIC(10, 3),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS detections (
  id BIGSERIAL PRIMARY KEY,
  detection_id TEXT NOT NULL UNIQUE,
  video_ref BIGINT NOT NULL REFERENCES videos(id) ON DELETE CASCADE,
  frame_id INTEGER NOT NULL CHECK (frame_id >= 0),
  video_timestamp_sec NUMERIC(10, 3) NOT NULL CHECK (video_timestamp_sec >= 0),
  confidence NUMERIC(4, 3) NOT NULL CHECK (confidence >= 0 AND confidence <= 1),
  latitude NUMERIC(10, 7) NOT NULL CHECK (latitude BETWEEN -90 AND 90),
  longitude NUMERIC(10, 7) NOT NULL CHECK (longitude BETWEEN -180 AND 180),
  gps GEOGRAPHY(POINT, 4326) NOT NULL,
  image_path TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS detections_video_time_idx
  ON detections (video_ref, video_timestamp_sec, frame_id);
CREATE INDEX IF NOT EXISTS detections_video_confidence_idx
  ON detections (video_ref, confidence);
CREATE INDEX IF NOT EXISTS detections_frame_idx
  ON detections (video_ref, frame_id);
CREATE INDEX IF NOT EXISTS detections_gps_idx
  ON detections USING GIST (gps);
```

Appendix B: Detection Reviews Migration

Source: sql/migrations/002_detection_reviews.sql

```sql
ALTER TABLE detections
  ADD COLUMN IF NOT EXISTS review_status TEXT
            CHECK (review_status IS NULL OR review_status IN ('approved', 'rejected'));

ALTER TABLE detections
  ADD COLUMN IF NOT EXISTS reviewed_at TIMESTAMPTZ;
```

Appendix C: Composition Root

Source: src/app.ts

The composition root constructs all dependencies, wires middleware and routes, and returns a configured Express application instance. The optional overrides parameter is the testing seam used by the Supertest-based integration tests.

```
export function createApp(overrides: AppOverrides = {}): express.Express {
  const { service, dynamicRepo } = buildDependencies(overrides);
  const detectionsController = createDetectionsController(service);
  const pipelineWorkflowController = createPipelineWorkflowController(
  getPipelineWorkflowStore(process.cwd()),
    dynamicRepo,
    config.mediaBaseUrl,
  );
  const authController = createAuthController({
    adminUsername: config.adminUsername,
    adminPassword: config.adminPassword,
    cookieName: config.authCookieName,
  });

  const app = express();
  app.use(express.json());
  app.use(session({
              name: config.authCookieName,
    secret: config.sessionSecret,
    resave: false,
    saveUninitialized: false,
    cookie: { httpOnly: true, sameSite: "lax",
              secure: config.env === "production",
              maxAge: config.sessionMaxAgeMs },
  }));

  app.get("/", (_req, res) => res.sendFile(...));
  app.get("/login", redirectAuthedAdmin, (_req, res) => res.sendFile(...));
  app.get("/map", (_req, res) => res.sendFile(...));
  app.get("/dashboard", requireAdminPage, (_req, res) => res.sendFile(...));

  app.use(createRouter({
    healthController, detectionsController, authController,
    mockWorkflowController, pipelineWorkflowController,
  }));
  app.use("/media", express.static(config.mediaRootPath));
  app.use(express.static(config.frontendPublicPath));
  app.use(notFoundHandler);
  app.use(errorHandler);
  return app;
}
```

Appendix D: Detection Service (excerpt)

Source: src/services/detection-service.ts

```
export class DetectionService {
  constructor(
    private readonly repository: DetectionRepositoryContract,
    private readonly mediaBaseUrl: string,
  ) {}

  async listDetections(filters: DetectionListQuery) {
            const result = await this.repository.listDetections(filters);
            return {
      items: result.rows.map((row) => this.serializeDetection(row)),
      total: result.total,
      limit: filters.limit,
      offset: filters.offset,
            }; }
  async getMapData(filters: DetectionListQuery) {
            const rows = await this.repository.getMapData(filters);
            const grouped = new Map<string, MapPoint[]>();
            for (const row of rows) {
      const points = grouped.get(row.video_id) ?? [];
      points.push({
        frame_id: row.frame_id,
        video_timestamp_sec: Number(row.video_timestamp_sec),
        gps: { latitude: Number(row.latitude),
              longitude: Number(row.longitude) },
        detection_count: Number(row.detection_count),
        max_confidence: Number(row.max_confidence),
        image_path: row.image_path,
        image_url: buildImageUrl(row.image_path, this.mediaBaseUrl),
            });
      grouped.set(row.video_id, points);
            }
            return { videos: Array.from(grouped.entries())
          .map(([video_id, points]) => ({ video_id, points })) };
  }
  async setReview(detectionId: string,
                decision: "approved" | "rejected") {
            const row = await this.repository.setReview(detectionId, decision);
            if (!row) {
      throw new HttpError(404, "detection_not_found", "Detection not found");
            }
            return this.toReviewApiPayload(row);
  }
}
```

Appendix E: Auth Controller — Timing-Safe Credential Check

Source: src/controllers/auth-controller.ts

Excerpt showing the constant-time string comparison used to defend against timing attacks during authentication, and the session regeneration to prevent fixation.

```
function secureEquals(left: string, right: string): boolean {
  const a = Buffer.from(left, "utf8");
  const b = Buffer.from(right, "utf8");
  if (a.length !== b.length) return false;
  return crypto.timingSafeEqual(a, b);
}
async login(req, res) {
  const credentials = validateLoginBody(req.body);
  const validUser = secureEquals(credentials.username, options.adminUsername);
  const validPass = secureEquals(credentials.password, options.adminPassword);
  if (!validUser || !validPass) {
            throw new HttpError(401, "invalid_credentials",
     "Invalid username or password");
  }
  await regenerateSession(req);   // prevent session fixation
  req.session.adminUser = { username: options.adminUsername, role: "admin" };
  await saveSession(req);
  res.json({ success: true, redirectTo: "/map" });}
```

Appendix F: Test Suite Excerpt

Source: test/app.test.ts

Sample test cases demonstrating the FakeService pattern and HTTP-level assertions used to validate the full middleware stack.

```
class FakeService {
  async listDetections(filters: DetectionListQuery) {
            return {
     items: [{ detection_id: "2411aabd", video_id: "run_001",
              frame_id: 241, video_timestamp_sec: 4.02,
              gps: { latitude: 26.222426, longitude: 44.135455 },
              confidence: 0.78,
              image_path: "runs/inference/run_001/frames/frame_0241.jpg",
              image_url: "http://localhost:3000/media/...frame_0241.jpg" }],
     total: 1, limit: filters.limit, offset: filters.offset,
            };
  }
  // ... other contract methods }

test("TC-05: filters detections by min confidence", async () => {
  const app = createApp({ service: new FakeService() });
  const res = await request(app)
    .get("/api/v1/detections?minConfidence=0.5")
    .expect(200);
  for (const item of res.body.items) {
    assert.ok(item.confidence >= 0.5);
  } } );
test("TC-10: frame endpoint returns 404 for unknown frame", async () => {
  const app = createApp({ service: new FakeService() });
  await request(app)
    .get("/api/v1/frames/run_999/9999")
```

```
    .expect(404)
    .expect((res) => assert.equal(res.body.code, "not_found"));
});
```

Appendix G: Video Inference Loop (run_video_inference)

Source: pipeline/scripts/inference.py

```
while True:
            ret, frame = cap.read()
            if not ret:
        break
    frame_idx += 1

            if skip > 1 and frame_idx % skip != 1:
        writer.write(frame)
        continue
    results = model.predict(
        source=frame, conf=args.conf, iou=args.iou,
        device=device, half=torch.cuda.is_available(),
        save=False, verbose=False,)

            r = results[0]
    annotated = r.plot()
    writer.write(annotated)

            if r.boxes is not None and len(r.boxes) > 0:
        timestamp_sec = round(frame_idx / fps, 2)
        cv2.imwrite(frame_path, annotated)
        for box in r.boxes:
            detections.append({
                "detection_id": uuid.uuid4().hex[:8],
                "video_id": args.name,
                "frame_id": frame_idx,
                "video_timestamp_sec": timestamp_sec,
                "gps": None,
                "confidence": round(box.conf.item(), 2),
                "image_path": frame_path,})

    print(f"PROGRESS:{processed}/{total_to_process}", flush=True)
            with open(json_path, "w") as f:
        json.dump(detections, f, indent=2)
```

Appendix H: GPS Coordinate Parser (_parse_segments)

Source: pipeline/scripts/ocr_gps.py

Implements Algorithm 2. Parses multi-segment EasyOCR output into a valid geographic coordinate by stripping noise, concatenating digits, and testing two- and three-digit integer parts.

```
def _parse_segments(segments):
    """ EasyOCR reads the OSD as separate text blocks, e.g.:
      ['LON', '4 4', '1 3 5 4 6 1 6']  -> 44.1354616
    Strategy: strip the label, join the digit segments, and split
              into integer part (2-3 digits) and decimal part."""
    digit_segments = []
              for seg in segments:
        cleaned = re.sub(r"[^0-9. ]", "", seg)
              if cleaned.strip():
            digit_segments.append(cleaned)
              if not digit_segments:
        return None
              digits = "".join(digit_segments).replace(" ", "").replace(".", "")
              if len(digits) < 6:
        return None
              for int_len in [2, 3]:
              if len(digits) > int_len:
            try:
                val = float(f"{digits[:int_len]}.{digits[int_len:]}")
            except ValueError:
                continue
            if -180 <= val <= 180:
                return val
              return None
```

Appendix I: Dynamic Detection Repository (Hot-Swap Proxy)

Source: src/repositories/dynamic-detection-repository.ts

Implements Algorithm 3. A thin proxy that forwards every repository method to an inner instance and exposes a single swap() operation for atomic data replacement without server restart.

```
export class DynamicDetectionRepository
  implements DetectionRepositoryContract {

  constructor(private inner: DetectionRepositoryContract) {}

  swap(next: DetectionRepositoryContract): void {
    this.inner = next;
  }

  listDetections(filters: DetectionListQuery)
              : Promise<{ rows: DetectionRow[]; total: number }> {
              return this.inner.listDetections(filters);
  }

  getMapData(filters: DetectionListQuery): Promise<MapRow[]> {
              return this.inner.getMapData(filters);
  }
```

```
  // ... other contract methods all delegate to this.inner
}
```

Appendix J: Pipeline Process Manager (runPipelineAsync)

Source: src/services/pipeline-workflow-store.ts

```
private async runPipelineAsync(
  job: PipelineJob, skipFrames: number,
): Promise<void> {
  job.status = "processing";
  job.progress = 0;
  const scriptPath = path.join(this.projectRoot,
    "pipeline", "scripts", "inference.py");
  const outputDir = path.join(this.projectRoot,
    "pipeline", "outputs", job.runName);
  await fs.promises.mkdir(outputDir, { recursive: true });
  await new Promise<void>((resolve, reject) => {
            const proc = spawn("python", [
      scriptPath, job.filePath,
      "--output", outputDir,
      "--skip-frames", String(skipFrames),
            ]);
    proc.stdout?.on("data", (data: Buffer) => {
      const out = data.toString();
      const m = out.match(/progress[:\s]+(\d+)/i);
            if (m) job.progress = Math.min(100, parseInt(m[1], 10));
            });
    proc.on("close", (code: number | null) => {
            if (code === 0) {
        const f = path.join(outputDir, "detections.json");
            if (fs.existsSync(f)) job.detectionsPath = f;
        job.progress = 100;
        job.status = "complete";
        resolve();
            } else {
        reject(new Error(`Pipeline exited with code ${code}`));
            }  });   }); }
```

Appendix K: Status Polling with Repository Swap (getUploadStatus)

Source: src/controllers/pipeline-workflow-controller.ts

On each status poll, checks whether the pipeline has completed. If so, constructs a new file-backed repository from the output JSON and hot-swaps it into the dashboard data layer.

```
async getUploadStatus(req, res, next) {
  const { uploadId } = req.params;
  const job = store.getJob(uploadId);
  if (!job) {
            throw new HttpError(404, "upload_not_found", "Upload not found");
  }

  // When the pipeline finishes, hot-swap the repository so the
  // dashboard shows the new run's data without a server restart.
  if (job.status === "complete" && job.detectionsPath
     && dynamicRepo && !job._swapped) {
            try {
     const newRepo = new SampleDetectionRepository(
       job.detectionsPath,
            );
     dynamicRepo.swap(newRepo);
     job._swapped = true;
            } catch {
            // If swap fails (e.g. file not yet flushed), retry next poll
            }
  }
  res.json({
    uploadId: job.uploadId,
    runName: job.runName,
    status: job.status,
    progress: job.progress,
            error: job.error,
  } ) ;}
```

MIRQAB

A Smart Potholes Detection System For Drone Footage

Rayan Mazen Alharbi 444000557
Abdullah Rajeh Alshehri 444001478
Ziyad Abdulaziz Almoneef 444003721
Mazen Saleh AlAbdullatif 444001320

Dr. Saad Alabbad

INTRODUCTION

- Manual road inspection is slow, expensive, and often unreliable.
- Potholes affect road safety and increase maintenance costs.
- Existing systems often lack accurate GPS-based localization.
- MIRQAB uses drones, AI, and GPS integration for automated detection and reporting.

OBJECTIVES

- Detect potholes automatically using deep learning
- Capture road footage using drones
- Extract GPS coordinates for each detection
- Visualize detections on an interactive dashboard
- Reduce manual inspection effort and improve road maintenance efficiency

METHODOLOGY

1. Drone captures road footage
2. Frames extracted from video
3. YOLOv8 detects potholes
4. OCR extracts GPS coordinates
5. Data stored in database
6. Dashboard visualizes detections

SYSTEM ARCHITECTURE

1. DRONE SUBSYSTEM

Drone Camera → Video Acquisition → GPS Module

2. AI DETECTION PIPELINE

YOLOv8 Object Detection → Frame Analysis → OCR GPS Extraction

3. BACKEND & DATABASE

Detection Storage → REST API → PostgreSQL + PostGIS

4. DASHBOARD

Interactive Map | Detection Review | Statistics & Reports

TECHNOLOGIES USED

Category	Technology
AI Model	YOLOv8
OCR	EasyOCR
Backend	Node.js + Express
Frontend	HTML / CSS / JavaScript
Database	PostgreSQL + PostGIS
Mapping	Leaflet
Language	Python & TypeScript

FEATURES

- Drone captures road footage
- AI-powered Detection
- GPS Geolocation
- Interactive Dashboard
- Exportable Reports
- Detection Validation

RESULTS

- Accurate pothole detection achieved.
- GPS coordinates linked successfully.
- Dashboard displays potholes interactively.
- Real-time upload and processing workflow implemented.
- System supports scalable road monitoring.

DASHBOARD – MAP VIEW

DASHBOARD – ANALYTICS VIEW

CHALLENGES

- OCR accuracy under motion blur
- GPS extraction reliability
- Long inference processing time
- Handling large drone videos

FUTURE WORK

- Real-time drone streaming
- Multi-drone support
- Mobile application integration
- Improved GPS precision
- Advanced road damage classification

CONCLUSION

MIRQAB provides a smart and scalable solution for automated road inspection using drones and artificial intelligence. The system reduces manual effort, improves detection efficiency, and supports safer road maintenance operations.